HANDS-ON | PRODUCTION-STYLE | ITSM TICKET-DRIVEN

LINUX CHALLENGES SIMULATOR

20 CHALLENGES



Break it. Debug it. Fix it.

20 real-world ITSM tickets • one bash simulator • RHEL, Ubuntu & SUSE command variants

Shaamnatheshwar Sasikumar

DevOps Engineer

About this simulator

The **Linux Challenges Simulator** is a hands-on, bash-driven practice lab with **20 production-style challenges**. Every challenge arrives the way work arrives in a real IT operations team: as an **ITSM ticket** - an Incident, a Service Request (RITM), a Change Request or a Problem record - with a ticket number, priority, caller, assignment group and affected configuration item. You pick it up, work it on a Linux VM, and close it with resolution notes.

These are **challenges for testing your own skills** against realistic operational situations: a full disk that is not really full, a service that is listening but unreachable, a script that fails only on certain mount points, a server crawling under swap pressure.

A single script, `linux_challenge.sh`, builds the workspace for any challenge on demand: the starter files and logs for the scenario, plus a `PROBLEM.md` that holds the full ticket with all its hints. The solution lives in a separate answer key (`linux_challenge_answer.md`) that you open only after a genuine attempt.

WHAT WE ARE DOING - THE 4-STEP LOOP

1 — Generate
Run `./linux_challenge.sh <N>` - the scenario files and `PROBLEM.md` appear under `~/linux-challenges/challengeN/`.

2 — Pick up the ticket
Read the ticket and `PROBLEM.md`: priority, what is broken or requested, and the work required.

3 — Work it
Solve it on a real Linux VM (KodeKloud). Run the commands and verify with the tools the ticket points to.

4 — Resolve & compare
Write your resolution notes, then open the answer key and compare commands and reasoning.

HOW EVERY CHALLENGE PAGE IS LAID OUT

- **Ticket card** - ITSM record with a generated number, priority, state, caller / requester, assignment group and configuration item.
- **Description** and **The Question** - what is broken or requested (the ticket's own wording), and a summary of the work required.
- **Files, Location & How to Run** - the command, workspace path, every file the script creates, and the deliverable path.
- **Closure criteria** and **What the script prints** - what must be true to close the ticket, and the real output of the script.
- **Solution** - exactly as you would type it (`$` = command you type, grey italics = comments, plain text = output), then the **orange Ubuntu / Debian block** with the same task.
- **Resolution Information** and **Key Takeaways** - the close code and notes, then the "why".

Ticket numbers, people, teams and servers are fictional. **Real output:** everything the script prints, and the outputs of challenges 1 (chmod / ls steps), 10, 11, 15, 16, 17 and 18, were produced by actually running the commands against the files the script creates; unit files were checked with `systemd-analyze verify` and the sudoers line with `visudo -cf`. Outputs that need root, LVM, firewalld, yum or a real server (hostnames, UIDs, UUIDs, the fictional `libfoo` package) are representative. In the Ubuntu / Debian blocks, commands built only from GNU tools (stat, readlink, zcat, xargs, gzip, tar, findmnt, df) were run in the build environment; apt, ufw, zypper and other distro-specific commands were checked against current documentation but not executed - try them on your own VM.

Tested on Ubuntu 24.04: the corrected Ubuntu / Debian steps in Challenges 1, 2, 18 and 20 (the SSH drop-in, `backup.sh` and the disk / inode monitor). The Challenge 20 unit with `ExecReload` and file logging was only checked with `systemd-analyze verify`, not on a live systemd.

ITSM ticket types used

Each challenge is one of four record types, the same ones you meet in tools like ServiceNow.

Prefix	Record	Meaning	Challenges
INC	**Incident**	Something is broken or degraded - restore service first.	1, 3, 5, 7, 9, 11, 13, 16, 17, 19
RITM	**Requested Item**	A standard service request from the catalog (child tasks are `SCTASK`).	2, 8, 12, 18
CHG	**Change Request**	A planned modification: Standard (pre-approved), Normal (CAB) or Emergency (child tasks are `CTASK`).	4, 6, 14, 15, 20
PRB	**Problem**	Root cause of a recurring issue, tracked beyond the single incident.	10

Priority is shown as P1 - Critical, P2 - High, P3 - Moderate or P4 - Low. Incident priority comes from impact and urgency.

Where each challenge can run

Every challenge builds its files anywhere, but some solutions need a real VM with root. Use this map to plan your lab time.

Environment	Challenges	What you need
Any Linux shell	10, 11, 15, 16, 17, 18	Just bash and coreutils - run and test everything for real, no root needed.
Analysis from a snapshot	3, 7, 19	You read captured `ps`, `ss`, `uptime`, `free`, `vmstat` output and reason like an incident call; the commands themselves run anywhere.
Real VM with root / sudo	1, 2, 4, 5, 6, 8, 9, 12, 13, 14, 20	KodeKloud (or any) Linux VM with systemd, LVM tools, a package manager, firewalld, cron/at, sshd and `sudo`.

DISTROS COVERED

Each solution shows the **RHEL-family** commands first (RHEL, Rocky, Alma, CentOS Stream, Fedora). Every challenge then has an **orange Ubuntu / Debian block** with the same task (identical commands are marked as identical; the differences are spelled out), and the **Distro Command Map** at the end puts RHEL, Ubuntu / Debian and SUSE side by side. The script itself needs only bash and GNU tools, so it runs on all of them (not Alpine / BusyBox or macOS).

GROUND RULES

- Read the ticket and `PROBLEM.md` and write your **own** commands first - open the solution only afterwards.
- Explain **why**, not just what. Treat the **Closure criteria** on every challenge as your own scoring check.
- Write the deliverable file named in the ticket into the challenge's workspace folder.
- Re-run `./linux_challenge.sh clean <N>` and repeat any challenge you could not finish without peeking.
- The **Command Cheat Sheet** at the end lists every command used, challenge by challenge.

How to run it

The script is a plain Bash file. Everything is created under `~/linux-challenges/` - your real system is never touched, and you can wipe one challenge or all of them at any time. Running a challenge that already exists never overwrites your work.

STEP BY STEP

bash - setup

```
$ mkdir -p ~/linux-sim && cd ~/linux-sim           # any folder you like
$ cp /path/to/linux_challenge.sh .                 # put the simulator script here
$ chmod +x linux_challenge.sh                      # make it executable (once)
# run with ./linux_challenge.sh or bash linux_challenge.sh, not sh
$ ./linux_challenge.sh list                        # show all 20 challenge titles
Linux Challenges Simulator
Available challenges:
  1  File Permissions & Ownership - chmod, chown & Special Bits
  2  User & Group Management - useradd, sudoers & Least Privilege
  ...
  20 Capstone - Provision & Harden a Production Linux Server

Usage: linux_challenge.sh <1-20> | list | show <N> | clean <N|all>
```

bash - work a challenge

```
$ ./linux_challenge.sh 4                           # build workspace + print ticket (full output next page)
$ ./linux_challenge.sh show 4                      # print the ticket again any time (or: cat PROBLEM.md)
$ ls ~/linux-challenges/challenge4/                # scenario files created for you
CURRENT_PRACTICE.md  PROBLEM.md  billing-worker.service  worker.py
# ... work the ticket on your lab VM, write the deliverable file named in the ticket ...
$ ./linux_challenge.sh 4                           # running it again NEVER overwrites your work
Workspace for challenge 4 already exists - your work is untouched.
(Use './linux_challenge.sh clean 4' if you want a fresh start.)
$ ./linux_challenge.sh clean 4                     # wipe challenge 4 and start fresh
Wiped challenge 4.
```

bash - after you have solved it

```
$ grep -n '^## 5\. ' linux_challenge_answer.md                      # find challenge 5 in the answer key
$ sed -n '/^## 5\. /,/^## 6\. /p' linux_challenge_answer.md | less  # read ONLY that challenge's answer
```

COMMAND REFERENCE

Command	What it does
`./linux_challenge.sh list`	Show all 20 challenge titles (also the default with no argument).
`./linux_challenge.sh <1-20>`	Build the workspace for that challenge and print its ticket.
`./linux_challenge.sh show <1-20>`	Print an already-built ticket again.
`./linux_challenge.sh clean <1-20>`	Wipe just that challenge's workspace.
`./linux_challenge.sh clean all`	Wipe **everything** under `~/linux-challenges/` (asks you to type yes when run interactively).

Sample output

This is the complete, real output of `./linux_challenge.sh 11` - the ticket you see in your terminal, also saved as `PROBLEM.md`. The output for all 20 challenges is in `linux_challenge_sample_output.txt`.

```
bash - ./linux_challenge.sh 11

$ ./linux_challenge.sh 11
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 11 of 20
================================================================================
 INCIDENT   INC0041978                                    PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : Customers report failed billing charges - HTTP 500 on
                      /api/billing/charge
 State              : In Progress
 Opened             : 07-Oct-2026 08:42
 Caller             : Chloe Dubois (Customer Support)
 Contact type       : Email
 Impact             : 2 - Medium
 Urgency            : 1 - High
 Category           : Application / Errors
 Business service   : Billing Platform
 Configuration item : web-srv-02 (access.log)
 Target resolution  : 8 hours
 Assignment group   : IT Operations - L2
 Assigned to        : You (DevOps Engineer)
--------------------------------------------------------------------------------
 DESCRIPTION
   Customers report failed billing charges. You have a raw web-server
   access.log (a 10-line sample here - imagine millions of lines) and must
   answer several questions fast, using only the Unix pipeline, as if
   live-debugging over SSH during an incident.

 WORK REQUIRED
  1. Count how many requests returned 500.
  2. List the unique IPs that got a 500 on /api/billing/charge, with counts,
     most frequent first.
  3. Extract METHOD + PATH from every line, show each unique pair once with its
     count (descending).
  4. Use sed to redact the source IP on every line.
  5. Why awk rather than cut for question 3?
--------------------------------------------------------------------------------
 WORKSPACE   : ~/linux-challenges/challenge11/
 DELIVERABLE : ~/linux-challenges/challenge11/commands.txt
 NEEDS       : Runs anywhere (any Linux shell).
 CLOSE WHEN  : Every command is a real one-liner producing the exact output
               shown, and the IP list is sorted by count, descending.
================================================================================

Workspace:       ~/linux-challenges/challenge11/
Read the brief:  cat ~/linux-challenges/challenge11/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge11/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge11/
PROBLEM.md  access.log  commands.txt

$ cat ~/linux-challenges/challenge11/commands.txt
# CHALLENGE 11 DELIVERABLE: commands.txt
# Text Processing - grep, awk, sed & the Unix Pipeline
#
# Replace this template with your answer. Brief: cat ~/linux-challenges/challenge11/PROBLEM.md
#
# WORK REQUIRED (from the ticket):
#   1. Count how many requests returned 500.
#   2. List the unique IPs that got a 500 on /api/billing/charge, with counts,
#      most frequent first.
#   3. Extract METHOD + PATH from every line, show each unique pair once with its
#      count (descending).
```

```
bash - ./linux_challenge.sh 11
#   4. Use sed to redact the source IP on every line.
#   5. Why awk rather than cut for question 3?
#
# ---- start your answer below this line ----
```

The 20 challenges

Click any title to jump to that challenge. Each line shows the challenge number, its ITSM ticket number and the topic.

01 INC0041831 File Permissions & Ownership - chmod, chown & Special Bits 8

02 RITM0010505 User & Group Management - useradd, sudoers & Least Privilege 11

03 INC0041888 Process Management - ps, kill, Signals, nice & Job Control 14

04 CHG0030636 systemd Service Management - Writing a Custom Unit File 17

05 INC0041904 Disk & Filesystem Management - df, du, mount & /etc/fstab 20

06 CHG0030686 LVM - Extending a Logical Volume Without Downtime 23

07 INC0041939 Networking Fundamentals - ip, ss, DNS & Connectivity Troubleshooting 26

08 RITM0010549 Firewall Management - firewalld / iptables Least-Privilege Rules 29

09 INC0041960 Package Management - Repos, Dependency Conflicts & Pinning 32

10 PRB0040095 Shell Scripting Fundamentals - A Real Production-Style Script 35

11 INC0041978 Text Processing - grep, awk, sed & the Unix Pipeline 39

12 RITM0010568 Cron & Scheduled Tasks - crontab and systemd Timers 42

13 INC0041994 Log Management - journalctl, /var/log & logrotate 45

14 CHG0030734 SSH Configuration & Hardening - Key-Based Auth Done Right 48

15 CHG0030764 find & xargs - Safe, Efficient Batch File Operations 51

16 INC0042027 Symbolic & Hard Links - Inodes and What Can Go Wrong 54

17 INC0042080 Environment Variables & Shell Profiles - PATH and Login Shells 57

18 RITM0010581 Archiving & Compression - tar, gzip & a Real Backup Strategy 60

19 INC0042117 Performance Troubleshooting - Load Average, Memory & I/O 63

20 CHG0030778 Capstone - Provision & Harden a Production Linux Server 66

Command Cheat Sheet - quick reference for all 20 challenges 71

Distro Command Map - RHEL vs Ubuntu / Debian vs SUSE 78

CHALLENGE
01

File Permissions & Ownership - chmod, chown & Special Bits

chmod | chown | setuid | setgid | sticky bit | ls -ld

INCIDENT
INC0041831

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**Finance team gets "permission denied" on shared_reports; backup.sh permissions too open**		
STATE	In Progress	OPENED	05-Oct-2026 11:05
CALLER	Olivia Bennett (Finance)	CONTACT TYPE	Self-service portal
IMPACT	2 - Medium	URGENCY	2 - Medium
CATEGORY	Access / Permissions	BUSINESS SERVICE	Finance Reporting
CONFIGURATION ITEM	fin-reports-01	TARGET RESOLUTION	3 business days
ASSIGNMENT GROUP	Linux Platform Ops	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

Two separate access problems have landed on your desk on the same Linux server:

1. ~/linux-challenges/challenge1/shared_reports/ is a directory used by the WHOLE "finance" team (multiple different Linux user accounts) to drop CSV reports. Right now, whoever creates a file in there ends up as its sole owner, and teammates can't overwrite/delete each other's files, causing constant "permission denied" complaints and manual `sudo chmod` workarounds by whoever is on call that day.
2. ~/linux-challenges/challenge1/scripts/backup.sh needs to be runnable by ANY member of the "backup-operators" group, but must NOT be world-writable, and only the owning user should be able to edit it.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Explain **setuid**, **setgid** and the **sticky bit** in one plain sentence each.
2. Fix `shared_reports/` with **setgid + sticky** (group `finance`) - give exact commands and explain the leading digit of the numeric mode.
3. Fix `backup.sh` so the group can **execute but not modify** it - justify the exact numeric mode and why `755` is not the best answer.
4. Explain why a **setuid** bit on a binary (e.g. `/bin/bash`) is dangerous.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 1`
Workspace	`~/linux-challenges/challenge1/`

Files created	`~/linux-challenges/challenge1/PROBLEM.md`
	`~/linux-challenges/challenge1/scripts/backup.sh`
	`~/linux-challenges/challenge1/shared_reports/q3_report.csv`
	`~/linux-challenges/challenge1/SOLUTION_COMMANDS.txt`

Deliverable	`~/linux-challenges/challenge1/SOLUTION_COMMANDS.txt`
Needs	`sudo` for `chown` / `groupadd` (the `chmod` parts run anywhere).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

• `ls -ld shared_reports` must show `drwxrws--T` (or equivalent) reflecting BOTH the setgid and sticky bits correctly applied.
• You must correctly explain, in your own words, why BOTH bits are independently necessary for problem 1.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 1

```
$ ./linux_challenge.sh 1
================================================================================
 LINUX CHALLENGES SIMULATOR                                    CHALLENGE 1 of 20
================================================================================
 INCIDENT   INC0041831                                PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Finance team gets "permission denied" on shared_reports;
                      backup.sh permissions too open
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge1/
Read the brief:  cat ~/linux-challenges/challenge1/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge1/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge1/
PROBLEM.md  SOLUTION_COMMANDS.txt  scripts  shared_reports
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge1

```
$ cd ~/linux-challenges/challenge1
$ ls -ld shared_reports scripts/backup.sh          # BEFORE
-rwxrwxr-x 1 student student   71 Oct 11 13:13 scripts/backup.sh
drwxrwxr-x 2 student student 4096 Oct 11 13:13 shared_reports

# ---- Problem 1: shared directory for the finance team ----
$ sudo groupadd finance                            # skip if the group already exists
$ sudo chown :finance shared_reports               # group-own the directory
$ sudo chmod 3775 shared_reports                   # 3 = setgid(2) + sticky(1) ; 775 = rwxrwxr-x
$ ls -ld shared_reports
drwxrwsr-t 2 student finance 4096 Oct 11 13:13 shared_reports
# 's' in the group slot = setgid   |   't' in the others slot = sticky (lowercase = x is also set)

# Stricter variant - no access at all for "others" (this is the form the ticket's check shows):
$ sudo chmod 3770 shared_reports
$ ls -ld shared_reports
drwxrws--T 2 student finance 4096 Oct 11 13:13 shared_reports
# uppercase 'T' = sticky is set but "others" has no execute bit

# ---- Problem 2: backup.sh ----
$ sudo groupadd backup-operators
$ sudo chown root:backup-operators scripts/backup.sh
$ sudo chmod 750 scripts/backup.sh                 # owner rwx, group r-x, others none
$ ls -l scripts/backup.sh
-rwxr-x--- 1 root backup-operators 71 Oct 11 13:13 scripts/backup.sh
# 755 is not world-writable either, but it lets EVERY user run the script - 750 limits it to the group

# ---- Security audit: list every setuid file on the box ----
$ sudo find / -xdev -perm -4000 -type f 2>/dev/null
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/mount
# Save the exact commands above into SOLUTION_COMMANDS.txt  (the deliverable)
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 1

```
# chmod, chown, groupadd and find -perm are GNU tools - identical on Ubuntu / Debian:
$ sudo groupadd finance                          # same  (Debian also has: sudo addgroup finance)
$ getent group finance                           # confirm the group exists - works on every distro
finance:x:1003:
$ sudo chown :finance shared_reports
$ sudo chmod 3775 shared_reports                 # setgid + sticky, exactly as in the main solution
$ stat -c '%A %a %U:%G %n' shared_reports         # exact mode, octal and owner in ONE line
drwxrwsr-t 3775 student:finance shared_reports
$ sudo groupadd backup-operators                 # skip if it already exists
$ sudo chown root:backup-operators scripts/backup.sh && sudo chmod 750 scripts/backup.sh
$ stat -c '%A %a %U:%G %n' scripts/backup.sh
-rwxr-x--- 750 root:backup-operators scripts/backup.sh
$ sudo find / -xdev -perm -4000 -type f 2>/dev/null         # setuid audit - list varies per distro
/usr/bin/passwd
/usr/bin/sudo
/usr/bin/su
/usr/bin/mount
/usr/bin/umount
/usr/bin/chsh
/usr/bin/gpasswd
# On Ubuntu / Debian the group that may use sudo is called "sudo" (RHEL: "wheel") -
# never add the finance or backup users to it for this ticket.
```

RESOLUTION INFORMATION

STATE	Resolved	CLOSE CODE	Solved (Permanently)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	05-Oct-2026 12:44
CLOSE NOTES	shared_reports/ re-owned by group `finance` with setgid + sticky (`3775`, or `3770` where others must have no access). `backup.sh` set to `750` root:backup-operators. Verified with `ls -ld`.		

KEY TAKEAWAYS

- **setuid** (executable): runs with the **file owner's** privileges. **setgid** (directory): new files inherit the **directory's group**. **sticky** (directory): only the file owner (or root) can delete or rename a file.
- setgid solves *shared group ownership*; sticky solves *"nobody deletes my file"*. This scenario needs **both**.
- `750` scopes execute rights to the one intended group. `755` is not world-writable either, but it lets every other user on the box read and run the script.
- A setuid-root binary with any bug hands root to any user. `u+s` on `/bin/bash` = instant root shell for everyone - which is why audits run `find / -perm -4000` first.
- **Distro note:** `chmod`, `chown`, `groupadd` and `find -perm` are identical on RHEL, Ubuntu / Debian and SUSE (GNU coreutils). Debian-family also ships friendlier `addgroup` / `adduser` wrappers.

CHALLENGE
02

User & Group Management - useradd, sudoers & Least Privilege

useradd | chage | /etc/sudoers.d | visudo | least privilege

SERVICE REQUEST (RITM)
RITM0010505

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**New hire onboarding: Linux account emma.h with scoped sudo for billing-app**		
STATE	Work in Progress	OPENED	05-Oct-2026 13:37
REQUESTED FOR	Emma Hartley (Junior DevOps Engineer)	REQUESTED BY	Daniel Whitfield (Hiring Manager)
CATALOG ITEM	New Hire - Linux Server Access	REQUEST	REQ0031079
APPROVAL	Approved (Manager)	CATALOG TASK	SCTASK0010573
CATEGORY	Access Management	BUSINESS SERVICE	Linux Server Access
CONFIGURATION ITEM	lab-vm-01	TARGET FULFILMENT	3 business days
ASSIGNMENT GROUP	Linux Platform Ops	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

A new engineer, Emma, is starting today. See ~/linux-challenges/challenge2/ONBOARDING_REQUEST.md for the exact access requirements from her manager and security team. Previous onboardings on this team have been sloppy - at least one past new hire was accidentally given FULL passwordless root sudo "just to be safe and save time," which a later audit flagged as a serious over-grant.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Write the `useradd` command and explain `-g` **vs** `-G`.
2. Force a password change on first login with `chage` and explain what it changes in `/etc/shadow`.
3. Write a **least-privilege sudoers entry** under `/etc/sudoers.d/` with full command paths.
4. Explain why "just give full sudo, we trust them" is a bad default.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 2`
Workspace	`~/linux-challenges/challenge2/`

Files created	`~/linux-challenges/challenge2/PROBLEM.md`
	`~/linux-challenges/challenge2/ONBOARDING_REQUEST.md`
	`~/linux-challenges/challenge2/useradd_command.txt`
	`~/linux-challenges/challenge2/sudoers_emma`

Deliverable	`~/linux-challenges/challenge2/useradd_command.txt` `~/linux-challenges/challenge2/sudoers_emma`
Needs	root / `sudo` on a Linux VM (a group called `developers` must exist).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

• Your sudoers entry must be scoped to EXACTLY the 2 commands listed, with full paths, and placed in `/etc/sudoers.d/`, never directly in `/etc/sudoers`.
• You must correctly distinguish `-g` vs `-G` in your `useradd` command.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 2

```
$ ./linux_challenge.sh 2
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 2 of 20
================================================================================
 SERVICE REQUEST (RITM)   RITM0010505                 PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : New hire onboarding: Linux account emma.h with scoped
                      sudo for billing-app
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge2/
Read the brief:  cat ~/linux-challenges/challenge2/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge2/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge2/
ONBOARDING_REQUEST.md  PROBLEM.md  sudoers_emma  useradd_command.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge2

```
$ head -5 ~/linux-challenges/challenge2/ONBOARDING_REQUEST.md
New hire onboarding ticket
==========================
Username:        emma.h
Full name:       Emma Hartley
Role:            Junior DevOps Engineer

$ sudo groupadd developers                          # only if it does not exist yet
$ sudo useradd -m -d /home/emma.h -s /bin/bash -G developers -c "Emma Hartley" emma.h
# -G = SECONDARY groups (add to developers).  -g would REPLACE her primary group - wrong here.
$ id emma.h
uid=1002(emma.h) gid=1002(emma.h) groups=1002(emma.h),1001(developers)

$ sudo passwd emma.h                                # set the temporary password
New password:
Retype new password:
passwd: password updated successfully
$ sudo chage -d 0 emma.h                            # last-change date = day 0 -> password already expired
$ sudo chage -l emma.h | head -1
Last password change                                     : password must be changed

# ---- least-privilege sudo: separate file, never edit /etc/sudoers directly ----
$ sudo visudo -f /etc/sudoers.d/emma-billing-app
emma.h ALL=(root) NOPASSWD: /usr/bin/systemctl restart billing-app, /usr/bin/journalctl -u billing-app
$ sudo chmod 0440 /etc/sudoers.d/emma-billing-app
$ sudo visudo -cf /etc/sudoers.d/emma-billing-app
/etc/sudoers.d/emma-billing-app: parsed OK

$ sudo -l -U emma.h                                 # verify EXACTLY what she can do
User emma.h may run the following commands on lab-vm:
    (root) NOPASSWD: /usr/bin/systemctl restart billing-app, /usr/bin/journalctl -u billing-app
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 2

```
$ sudo groupadd developers                          # only if it does not exist yet
# useradd, usermod, passwd, chage and visudo behave the same on RHEL, Ubuntu / Debian and SUSE:
$ sudo useradd -m -d /home/emma.h -s /bin/bash -G developers -c "Emma Hartley" emma.h
$ sudo usermod -aG developers emma.h                # add to a group later (-a = APPEND, never forget it)
$ sudo chage -d 0 emma.h                            # force the password change - identical everywhere
$ sudo visudo -cf /etc/sudoers.d/emma-billing-app  # validate the scoped sudoers file - same everywhere
# Debian-family also ships "adduser", an interactive wrapper. Its default username rule can reject a dotted
# name like emma.h, so the portable useradd above is the safer choice for this ticket.
# The admin group is named differently: RHEL = wheel, Ubuntu / Debian = sudo.
# Emma gets NEITHER - only her two-command file in /etc/sudoers.d/.
```

RESOLUTION INFORMATION

STATE	Resolved	CLOSE CODE	Request fulfilled
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	05-Oct-2026 16:22
CLOSE NOTES	Account `emma.h` created with `-G developers`; password expired at first login with `chage -d 0`; scoped sudoers entry in `/etc/sudoers.d/` validated with `visudo -cf`.		

KEY TAKEAWAYS

- `-g` = primary group (exactly one). `-G` = supplementary groups (many). Emma needs `-G developers`.
- `chage -d 0` sets the "last password change" field of `/etc/shadow` to day 0, so the system treats the password as expired and forces a change at next login.
- Use **full paths** in sudoers (`/usr/bin/systemctl`) - rules match the exact executable, so a manipulated `PATH` cannot swap in another binary.
- Files in `/etc/sudoers.d/` are isolated and reviewable; a typo cannot corrupt the main sudoers file. Always validate with `visudo -cf`.
- Least privilege limits the **blast radius**: a phished password or leaked key can only restart one service, not own the box.
- **Distro note:** the full-sudo group is `wheel` on RHEL but `sudo` on Ubuntu / Debian - adding Emma to either would be the over-grant this ticket warns about. The scoped file in `/etc/sudoers.d/` is identical on every distro.

CHALLENGE

03

Process Management - ps, kill, Signals, nice & Job Control

ps | kill | SIGTERM / SIGKILL | renice | bg / fg / disown | nohup

INCIDENT

INC0041888

PRIORITY

P2 - High

SHORT DESCRIPTION	**worker.py processes stuck in R / Z / D states on app-srv-03, CPU pegged at 97%**		
STATE	In Progress	OPENED	05-Oct-2026 14:36
CALLER	Monitoring alert: CPU > 95% for 30 min	CONTACT TYPE	Monitoring alert
IMPACT	2 - Medium	URGENCY	1 - High
CATEGORY	Application / Performance	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03	TARGET RESOLUTION	8 hours
ASSIGNMENT GROUP	IT Operations - L2	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

An application server running several `worker.py` processes is in trouble - see the captured `ps` snapshot (with a PPID column) in ~/linux-challenges/challenge3/PS_OUTPUT_SNAPSHOT.md. You've been asked to triage this LIVE, explaining your reasoning at each step, as if this were a real incident call.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Explain the STAT values **R, S, Z, D** - and why `kill -9` on a **D** process is dangerous.
2. What is a **zombie**, why will `kill -9 4901` not remove it, and what actually cleans it up? (The PPID column shows the parent.)
3. **SIGTERM vs SIGKILL** on PID 4471 - which first and why?
4. Lower PID 4471's priority with `renice` and explain the niceness range.
5. A colleague must disconnect SSH without killing a foreground job - walk through `Ctrl+Z`, `bg`, `jobs`, `disown`, and SIGHUP.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 3`
Workspace	`~/linux-challenges/challenge3/`

Files created	`~/linux-challenges/challenge3/PROBLEM.md`
	`~/linux-challenges/challenge3/PS_OUTPUT_SNAPSHOT.md`
	`~/linux-challenges/challenge3/ANSWER.md`

Deliverable	`~/linux-challenges/challenge3/ANSWER.md` (written explanation + commands)
Needs	Analysis challenge - the commands run on any Linux shell.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- You must correctly distinguish zombie vs. uninterruptible-sleep processes, and correctly explain why killing a zombie directly does nothing.
- You must correctly state the SIGTERM-before-SIGKILL escalation order and justify it.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 3

```
$ ./linux_challenge.sh 3
================================================================================
 LINUX CHALLENGES SIMULATOR                                    CHALLENGE 3 of 20
================================================================================
 INCIDENT   INC0041888                                  PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : worker.py processes stuck in R / Z / D states on
                      app-srv-03, CPU pegged at 97%
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge3/
Read the brief:  cat ~/linux-challenges/challenge3/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge3/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge3/
ANSWER.md  PROBLEM.md  PS_OUTPUT_SNAPSHOT.md
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge3

```
$ cd ~/linux-challenges/challenge3
$ sed -n '3,8p' PS_OUTPUT_SNAPSHOT.md
USER  PID  PPID %CPU %MEM    VSZ   RSS TTY STAT START   TIME COMMAND
app  4400     1  0.0  0.1  98000  4100 ?   Ss   08:02   0:00 /usr/bin/python3 /opt/app/supervisor.py
app  4471  4400 97.2  1.1 812004 45120 ?   R    08:03  41:12 /usr/bin/python3 /opt/app/worker.py
app  4472  4400  0.1  0.3 221004 12300 ?   S    08:03   0:02 /usr/bin/python3 /opt/app/worker.py
app  4901  4472  0.0  0.0      0     0 ?   Z    09:14   0:00 [worker.py] <defunct>
app  5002  4400  1.9  1.0 801200 44000 ?   D    09:40   6:03 /usr/bin/python3 /opt/app/worker.py

# R = running   S = interruptible sleep   Z = zombie   D = uninterruptible sleep (waiting on I/O)

# ---- the zombie (PID 4901): its PARENT must reap it - the PPID column says 4472 ----
$ ps -o ppid= -p 4901                              # on a live system
4472
$ sudo kill -15 4472                               # ask the parent to exit; init then reaps the zombie

# ---- PID 4471: graceful first, force only if needed ----
$ sudo kill -15 4471                               # SIGTERM: app can flush, close DB handles, exit cleanly
$ sleep 10; ps -p 4471 >/dev/null && sudo kill -9 4471     # SIGKILL only if it is STILL alive

# ---- lower its CPU priority instead of killing it ----
$ sudo renice -n 10 -p 4471
4471 (process ID) old priority 0, new priority 10
# range: -20 (highest priority) ... +19 (lowest).  HIGHER number = NICER = LOWER priority.

# ---- PID 5002 (D state): find out WHY before touching it ----
$ iostat -x 1 3                                    # is the disk saturated?
$ sudo dmesg -T | tail                             # I/O errors, NFS timeouts?

# ---- a foreground job must survive SSH logout ----
$ ./long_job.sh
^Z
[1]+  Stopped                 ./long_job.sh
$ bg %1                                            # resume it in the background
[1]+ ./long_job.sh &
$ jobs -l                                          # confirm it is running
$ disown -h %1                                     # shell will no longer send it SIGHUP on exit
# from the start:  nohup ./long_job.sh > job.log 2>&1 &
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 3

```
# ps, kill, renice, jobs, bg and disown are the same everywhere. iostat is NOT installed by default:
$ sudo dnf install -y sysstat                    # RHEL / Rocky / Alma / Fedora
$ sudo apt install -y sysstat                    # Ubuntu / Debian
$ sudo zypper install -y sysstat                 # SUSE / openSUSE
$ iostat -x 1 3                                  # same output on all of them
```

RESOLUTION INFORMATION

STATE	Resolved	CLOSE CODE	Solved (Work Around)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	05-Oct-2026 16:28
CLOSE NOTES	Zombie parent identified from the PPID column and restarted; PID 4471 reniced then stopped with SIGTERM before SIGKILL; D-state process investigated with `iostat` and `dmesg` before any action.		

KEY TAKEAWAYS

- A **zombie is not running** - it is a finished process whose exit status the parent has not collected with `wait()`. There is nothing left to kill; fix or restart the **parent** (PPID).
- A **D-state** process is inside a kernel I/O call that signals cannot interrupt. Forcing it risks corruption - investigate the storage layer first.
- **SIGTERM before SIGKILL**: SIGTERM can be caught (graceful cleanup); SIGKILL cannot, and may leave locks, half-written files and open transactions.
- Closing SSH sends **SIGHUP** to attached processes. `nohup` ignores it from the start; `disown` detaches an existing job from the shell.
- **Distro note:** the process tools come from `procps` on every distro; only `iostat` needs an extra package (`sysstat`).

CHALLENGE
04

systemd Service Management - Writing a Custom Unit File

systemd unit | Restart= | User= | enable vs start | daemon-reload

CHANGE REQUEST
CHG0030636

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**Convert billing-worker from a manually started script into a managed systemd service**		
STATE	Implement	OPENED	05-Oct-2026 15:37
TYPE	Normal	RISK	Moderate
REQUESTED BY	Lucas Fontaine (Application Support)	APPROVAL	Approved (CAB)
CATEGORY	Service Management	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03	CHANGE TASK	CTASK0030923
PLANNED START	06-Oct-2026 22:00	PLANNED END	07-Oct-2026 00:00
ASSIGNMENT GROUP	Cloud & DevOps	ASSIGNED TO	You (DevOps Engineer)
JUSTIFICATION	The worker stayed down after 2 of the last 3 reboots and nobody was alerted. Managed service gives boot start, restart on failure and status visibility.		

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

The "billing-worker" Python process (~/linux-challenges/challenge4/worker.py) is currently run the way described in ~/linux-challenges/challenge4/CURRENT_PRACTICE.md - which is exactly why it's been silently NOT running after 2 of the last 3 server reboots, with nobody noticing for hours each time. You've been asked to turn this into a proper, managed systemd service.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Write `billing-worker.service` with `Description=` and `After=network.target`.
2. `[Service]`: full-path `ExecStart=`, `Restart=on-failure`, `RestartSec=5`, non-root `User=` / `Group=`.
3. `[Install]`: `WantedBy=multi-user.target` - and explain **enable vs start**.
4. Show install + verify: copy, `daemon-reload`, `enable --now`, `status`.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 4`
Workspace	`~/linux-challenges/challenge4/`

Files created	`~/linux-challenges/challenge4/PROBLEM.md`
	`~/linux-challenges/challenge4/CURRENT_PRACTICE.md`
	`~/linux-challenges/challenge4/worker.py`
	`~/linux-challenges/challenge4/billing-worker.service`

Deliverable	`~/linux-challenges/challenge4/billing-worker.service`
Needs	root + systemd (any systemd distro: RHEL-family, Ubuntu / Debian, SUSE).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- The unit file must include Restart=on-failure, a non-root User=, and WantedBy=multi-user.target under [Install].
- You must correctly explain the enable vs. start distinction and when daemon-reload is required.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 4

```
$ ./linux_challenge.sh 4
================================================================================
 LINUX CHALLENGES SIMULATOR                                    CHALLENGE 4 of 20
================================================================================
 CHANGE REQUEST   CHG0030636                        PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Convert billing-worker from a manually started script
                      into a managed systemd service
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge4/
Read the brief:  cat ~/linux-challenges/challenge4/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge4/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge4/
CURRENT_PRACTICE.md  PROBLEM.md  billing-worker.service  worker.py
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge4

```
$ sudo useradd -r -s /sbin/nologin billing         # dedicated non-root service account
$ sudo mkdir -p /opt/app && sudo cp ~/linux-challenges/challenge4/worker.py /opt/app/
$ sudo chown -R billing:billing /opt/app

$ cat > ~/linux-challenges/challenge4/billing-worker.service <<'EOF'
[Unit]
Description=Billing Worker Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/app/worker.py
Restart=on-failure
RestartSec=5
User=billing
Group=billing

[Install]
WantedBy=multi-user.target
EOF

$ systemd-analyze verify ~/linux-challenges/challenge4/billing-worker.service   # no output = OK
$ sudo cp ~/linux-challenges/challenge4/billing-worker.service /etc/systemd/system/
$ sudo systemctl daemon-reload                     # systemd caches units - tell it to re-read
$ sudo systemctl enable --now billing-worker       # enable = start on boot ; --now = start right now
Created symlink .../multi-user.target.wants/billing-worker.service → .../billing-worker.service.
$ systemctl status billing-worker --no-pager | head -3
● billing-worker.service - Billing Worker Service
     Loaded: loaded (/etc/systemd/system/billing-worker.service; enabled)
     Active: active (running)

# ---- prove Restart=on-failure works: simulate a crash ----
$ sudo kill -9 $(systemctl show -p MainPID --value billing-worker)
$ sleep 6; systemctl is-active billing-worker
active                                             # systemd restarted it after RestartSec=5
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 4

```
# The unit file and every systemctl command are identical on any systemd distro. Two small differences:
$ sudo useradd -r -s /usr/sbin/nologin billing     # Ubuntu / Debian path (RHEL: /sbin/nologin)
$ command -v python3                               # confirm the path you put in ExecStart=
/usr/bin/python3
$ sudo systemctl daemon-reload && sudo systemctl enable --now billing-worker
$ systemctl status billing-worker --no-pager | head -3
```

RESOLUTION INFORMATION

STATE	Closed - Successful	CLOSE CODE	Successful
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	07-Oct-2026 00:00
CLOSE NOTES	`billing-worker.service` installed under `/etc/systemd/system`, runs as non-root `billing` user with `Restart=on-failure`; enabled and crash-tested with `kill -9`.		

KEY TAKEAWAYS

- `After=network.target` is a cheap safe default: it avoids racing network start-up (DNS, DB, APIs) on a slow boot.
- Use the **full path** to `python3` - units run in a minimal environment where `PATH` may differ from your shell.
- `Restart=on-failure` restarts on crashes / non-zero exit, **not** on a clean `exit 0` and **not** after a deliberate `systemctl stop`. `RestartSec=5` prevents a CPU-burning crash loop.
- **Never run an app worker as root** just because it is easier - a compromised app then owns the machine.
- `start` = run now (this boot only). `enable` = create the `WantedBy` symlink so it starts every boot. `daemon-reload` is required after creating or editing any unit file.
- **Distro note:** only the no-login shell path differs (`/usr/sbin/nologin` on Ubuntu / Debian, `/sbin/nologin` on RHEL). Unit files and `systemctl` behave the same on every systemd distro.

CHALLENGE
05

Disk & Filesystem Management - df, du, mount & /etc/fstab

df -h | df -i | inodes | find | blkid | /etc/fstab | mount -a

INCIDENT
INC0041904

PRIORITY
P1 - Critical

SHORT DESCRIPTION	**"No space left on device" on / although df shows 600M free; new disk must be mounted permanently**		
STATE	In Progress	OPENED	05-Oct-2026 17:22
CALLER	Application error: "No space left on device"	CONTACT TYPE	Phone
IMPACT	1 - High	URGENCY	1 - High
CATEGORY	Storage	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03 (/dev/sda1)	TARGET RESOLUTION	4 hours
ASSIGNMENT GROUP	Linux Platform Ops	ASSIGNED TO	You (DevOps Engineer)
RELATED TASK	SCTASK0010580		

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

An application on this server just started throwing "No space left on device" errors - but see the `df -h` output in ~/linux-challenges/challenge5/DF_OUTPUT.md CAREFULLY, alongside the `df -i` (inode) output right below it, before you conclude what's actually wrong. Separately, the team also needs a new disk permanently mounted (see ~/linux-challenges/challenge5/FSTAB_CURRENT.md).

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Diagnose precisely: **space or inodes?** Explain what an inode is and a realistic cause of exhaustion.
2. Give the `find`-based approach to locate the directory that is eating the inodes.
3. Write the `/etc/fstab` entry using the **UUID**, and explain why UUID beats `/dev/sdb1`.
4. Show get-UUID, create mount point, add the line, `mount -a` - and why `mount -a` is also a validation step.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 5`
Workspace	`~/linux-challenges/challenge5/`

Files created	`~/linux-challenges/challenge5/PROBLEM.md`
	`~/linux-challenges/challenge5/DF_OUTPUT.md`
	`~/linux-challenges/challenge5/FSTAB_CURRENT.md`
	`~/linux-challenges/challenge5/diagnosis.txt`
	`~/linux-challenges/challenge5/fstab_entry.txt`

Deliverable	`~/linux-challenges/challenge5/diagnosis.txt` `~/linux-challenges/challenge5/fstab_entry.txt`
Needs	root on a VM with a spare ext4 partition for the fstab part.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- You must correctly identify whether this is a disk-space or inode problem based on the given df output, and justify it with the actual numbers.
- Your fstab entry must use a UUID, not a raw device path.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 5

```
$ ./linux_challenge.sh 5
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 5 of 20
================================================================================
 INCIDENT   INC0041904                              PRIORITY: P1 - Critical
--------------------------------------------------------------------------------
 Short description  : "No space left on device" on / although df shows 600M
                      free; new disk must be mounted permanently
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge5/
Read the brief:  cat ~/linux-challenges/challenge5/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge5/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge5/
DF_OUTPUT.md  FSTAB_CURRENT.md  PROBLEM.md  diagnosis.txt  fstab_entry.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge5

```
$ cd ~/linux-challenges/challenge5
# the figures below come from DF_OUTPUT.md (captured on app-srv-03)
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        20G   19G  600M  97% /

$ df -i /
Filesystem       Inodes  IUsed   IFree IUse% Mounted on
/dev/sda1       1310720 1310000     720   99% /
# 600M of BYTES free but only 720 INODES free -> inode exhaustion, not disk space.
# Millions of tiny files (sessions, cache fragments, log snippets) each burn one inode.

# ---- hunt the culprit: count entries per suspect directory (-xdev stays on this filesystem) ----
$ for d in /var/log /tmp /var/spool /var/lib; do echo "$(sudo find $d -xdev | wc -l) $d"; done | sort -rn
# the directory with millions of entries is your answer - drill down the same way:
$ sudo find /var/spool -xdev -type f | cut -d/ -f1-4 | sort | uniq -c | sort -rn | head -5

# ---- permanent mount of the new disk by UUID ----
$ sudo blkid /dev/sdb1
/dev/sdb1: UUID="3f2a9c1e-77b4-4d0e-9c58-2b6f0a1d4e90" TYPE="ext4"
$ sudo mkdir -p /opt/app-logs
$ sudo cp /etc/fstab /etc/fstab.bak               # always keep a way back
$ sudo tee -a /etc/fstab <<'EOF'
UUID=3f2a9c1e-77b4-4d0e-9c58-2b6f0a1d4e90  /opt/app-logs  ext4  defaults,nofail  0 2
EOF
# nofail: if this data disk is ever missing the server still boots instead of dropping into emergency mode
$ sudo mount -a                                   # no output = the new line is valid
$ findmnt /opt/app-logs
TARGET        SOURCE    FSTYPE OPTIONS
/opt/app-logs /dev/sdb1 ext4   rw,relatime
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 5

```
# df, du, inodes, blkid, /etc/fstab and mount -a are identical. Ubuntu / Debian extras:
$ lsblk -f                                          # UUID + filesystem of every disk (instead of blkid)
NAME   FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
sda
└─sda1 ext4   1.0         7c1e5b52-0d3e-4a63-9d4b-1f2a3b4c5d6e   600M    97% /
sdb
└─sdb1 ext4   1.0         3f2a9c1e-77b4-4d0e-9c58-2b6f0a1d4e90
# Ubuntu mounts every snap as a read-only squashfs "loop" at 100% used - hide them in df:
$ df -h -x tmpfs -x squashfs -x devtmpfs
$ df -i -x tmpfs -x squashfs -x devtmpfs            # same filter for the inode check

# ---- permanent mount of the new disk by UUID ----
$ sudo mkdir -p /opt/app-logs
$ sudo cp /etc/fstab /etc/fstab.bak
$ sudo tee -a /etc/fstab <<'EOF'
UUID=3f2a9c1e-77b4-4d0e-9c58-2b6f0a1d4e90  /opt/app-logs  ext4  defaults,nofail  0 2
EOF
# nofail: if this data disk is ever missing the server still boots instead of dropping into emergency mode
$ sudo findmnt --verify                             # checks fstab syntax / UUIDs WITHOUT mounting
Success, no errors or warnings detected
$ sudo systemctl daemon-reload                      # systemd turns fstab lines into mount units
$ sudo mount -a
$ findmnt /opt/app-logs
TARGET        SOURCE    FSTYPE OPTIONS
/opt/app-logs /dev/sdb1 ext4   rw,relatime
```

RESOLUTION INFORMATION

STATE	Resolved	CLOSE CODE	Solved (Permanently)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	05-Oct-2026 19:26
CLOSE NOTES	Inode exhaustion confirmed (99% inodes vs 97% space) and the culprit directory located; `/dev/sdb1` mounted by UUID in `/etc/fstab` and validated with `mount -a`.		

KEY TAKEAWAYS

- An **inode** stores a file's metadata and block pointers - every file and directory needs one. Run out and you cannot create even an empty file, no matter how many free bytes remain.
- Always read `df -h` **and** `df -i` before concluding "disk full". Here `/` is out of inodes (99%) while still showing 600M free.
- `/dev/sdX` names are assigned by detection order and can change after adding or removing disks; a **UUID** is written at format time and never changes.
- `mount -a` immediately **validates** the new fstab line. A typo found at the next reboot can drop the server into emergency mode.
- **Distro note:** `df`, `blkid`, `/etc/fstab` and `mount -a` are identical on RHEL, Ubuntu / Debian and SUSE. For a data disk that may be absent at boot, add `nofail` to the options so a missing disk cannot drop the server into emergency mode.

CHALLENGE
06

LVM - Extending a Logical Volume Without Downtime

PV / VG / LV | lvextend | resize2fs | pvcreate | vgextend

CHANGE REQUEST
CHG0030686

PRIORITY
P2 - High

SHORT DESCRIPTION	**Emergency change: extend /data logical volume online - 95% full and growing**		
STATE	Implement	OPENED	06-Oct-2026 08:49
TYPE	Emergency	RISK	Moderate
REQUESTED BY	Sophia Marchetti (Data Platform)	APPROVAL	Approved (Emergency CAB)
CATEGORY	Storage	BUSINESS SERVICE	Data Platform
CONFIGURATION ITEM	data-srv-01 (data_vg/app_data)	CHANGE TASK	CTASK0030971
PLANNED START	06-Oct-2026 09:49	PLANNED END	06-Oct-2026 11:49
ASSIGNMENT GROUP	Linux Platform Ops	ASSIGNED TO	You (DevOps Engineer)
JUSTIFICATION	/data is 95% full, the writing application cannot be paused, and data_vg still holds 20 GB of unallocated space.		

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

The `/data` filesystem (backed by LVM, see ~/linux-challenges/challenge6/LVM_CURRENT_STATE.md) is at 95% full and climbing - the application writing to it cannot pause or be easily cleaned up on short notice. Fortunately, the underlying Volume Group still has 20GB of FREE, UNALLOCATED space available - it was deliberately provisioned with headroom for exactly this situation.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Explain the **PV -> VG -> LV** model and why the extra layer makes this possible.
2. Give the exact commands: `lvextend` to use all free space **and** the matching `resize2fs`.
3. How do you verify the fix, and what does it mean if `df` still shows the old size?
4. What changes if the VG had **zero** free space? (`pvcreate` -> `vgextend` -> ...)

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 6`
Workspace	`~/linux-challenges/challenge6/`

Files created	`~/linux-challenges/challenge6/PROBLEM.md`
	`~/linux-challenges/challenge6/LVM_CURRENT_STATE.md`
	`~/linux-challenges/challenge6/lvm_commands.txt`

Deliverable	`~/linux-challenges/challenge6/lvm_commands.txt`
Needs	root on a VM with LVM tools (a spare disk for the "no free space" path). Ubuntu / Debian: `apt install lvm2`.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

• Your command sequence must include BOTH the `lvextend` step AND the matching `resize2fs` step - listing only one is an incomplete, wrong answer for this challenge.
• You must correctly explain why online grow works for ext4 but isn't symmetric with shrink operations.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

```
bash - ./linux_challenge.sh 6

$ ./linux_challenge.sh 6
================================================================================
 LINUX CHALLENGES SIMULATOR                                    CHALLENGE 6 of 20
================================================================================
 CHANGE REQUEST   CHG0030686                                PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : Emergency change: extend /data logical volume online -
                      95% full and growing
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge6/
Read the brief:  cat ~/linux-challenges/challenge6/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge6/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge6/
LVM_CURRENT_STATE.md  PROBLEM.md  lvm_commands.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

```
bash - challenge6

$ sudo vgs data_vg
  VG      #PV #LV #SN Attr   VSize    VFree
  data_vg   1   1   0 wz--n- 100.00g  20.00g       # 20 GB free in the pool
$ df -h /data
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/data_vg-app_data   79G   75G  4.0G  95% /data

# ---- grow the LV into ALL remaining free space ----
$ sudo lvextend -r -l +100%FREE /dev/data_vg/app_data   # -r also grows the filesystem
  Size of logical volume data_vg/app_data changed from 80.00 GiB to 100.00 GiB (25600 extents).
  Logical volume data_vg/app_data successfully resized.

# ---- grow the ext4 filesystem on top (online - /data stays mounted) ----
# (not needed after -r; ext4 only - RHEL 8/9 defaults to XFS, where resize2fs fails: use xfs_growfs /data)
$ sudo resize2fs /dev/data_vg/app_data
resize2fs 1.46.5 (30-Dec-2021)
Filesystem at /dev/data_vg/app_data is mounted on /data; on-line resizing required
The filesystem on /dev/data_vg/app_data is now 26214400 (4k) blocks long.

$ df -h /data                                       # verify
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/data_vg-app_data   98G   75G   18G  81% /data
# (shortcut: lvextend -r -l +100%FREE ...  grows the LV and the filesystem in one step)

# ---- if the VG had ZERO free space: add a new disk first ----
$ sudo pvcreate /dev/sdc                            # initialise the new disk for LVM
$ sudo vgextend data_vg /dev/sdc                    # add it to the pool
$ sudo lvextend -l +100%FREE /dev/data_vg/app_data   # add -r here too (as above)
$ sudo resize2fs /dev/data_vg/app_data               # ext4 only; skip with -r (XFS: xfs_growfs /data)
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 6

```
# LVM commands are identical on every distro. Only the package and the default filesystem differ.
$ sudo dnf install -y lvm2                          # RHEL-family (usually preinstalled)
$ sudo apt install -y lvm2                          # Ubuntu / Debian
$ sudo zypper install -y lvm2                       # SUSE
$ findmnt -no FSTYPE /data                          # ext4 -> resize2fs, xfs -> xfs_growfs
ext4
$ sudo resize2fs /dev/data_vg/app_data              # ext4  (Ubuntu / Debian installer default)
$ sudo xfs_growfs /data                             # XFS (RHEL 8/9 default) - takes the MOUNT POINT
$ sudo lvextend -r -l +100%FREE /dev/data_vg/app_data    # either filesystem: LV + filesystem in one step
# Ubuntu's installer names it /dev/mapper/ubuntu--vg-ubuntu--lv (double dashes) - read yours from lvs / df.
```

RESOLUTION INFORMATION

STATE	Closed - Successful	CLOSE CODE	Successful
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	06-Oct-2026 11:49
CLOSE NOTES	`lvextend -l +100%FREE` followed by `resize2fs` online; `/data` now 81% used with no downtime.		

KEY TAKEAWAYS

- **PV** = raw disk (the pipe), **VG** = pooled capacity (the reservoir), **LV** = flexible slice you format and mount (the tap). Growing an LV never needs repartitioning.
- `lvextend` alone is **not enough** - the LV is bigger, but ext4 still believes it is the old size until `resize2fs` runs.
- Growing is safe online because it only adds unused space at the end. **Shrinking** may have to relocate live data, so it usually requires an unmounted filesystem - grow and shrink are not symmetric.
- LV bigger but `df` unchanged = you forgot (or failed) the `resize2fs` step. For XFS (the RHEL 8/9 default) use `xfs_growfs /data` instead of `resize2fs`.
- **Distro note:** RHEL 8/9 defaults to XFS (`xfs_growfs <mountpoint>`), Ubuntu / Debian installers default to ext4 (`resize2fs <device>`). `lvextend -r` handles either, so it is the portable choice.

CHALLENGE
07

Networking Fundamentals - ip, ss, DNS & Connectivity Troubleshooting

ss -tlnp | ip addr | loopback vs 0.0.0.0 | nc | traceroute

INCIDENT
INC0041939

PRIORITY
P2 - High

SHORT DESCRIPTION	**billing-api on port 8080 returns "Connection refused" for every other server**		
STATE	In Progress	OPENED	06-Oct-2026 11:41
CALLER	Liam Prescott (Dev Team)	CONTACT TYPE	Phone
IMPACT	2 - Medium	URGENCY	1 - High
CATEGORY	Network	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	billing-api-01 (10.0.1.55)	TARGET RESOLUTION	8 hours
ASSIGNMENT GROUP	Network & Platform Support	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

"billing-api" is supposed to be reachable on port 8080 from OTHER servers on the same internal network - it isn't. See ~/linux-challenges/challenge7/SYMPTOM_DATA.md for the diagnostics ALREADY gathered before this was escalated to you.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Why does a process that **is** listening on 8080 still refuse other hosts? (read the Local Address column)
2. What is the fix - and why is it an **application** setting, not a network-tool fix?
3. After the fix, name **2 other layers** that can still block traffic and the commands to narrow it down.
4. `ss` vs `netstat` - why is `ss` the modern default?

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 7`
Workspace	`~/linux-challenges/challenge7/`

Files created	`~/linux-challenges/challenge7/PROBLEM.md`
	`~/linux-challenges/challenge7/SYMPTOM_DATA.md`
	`~/linux-challenges/challenge7/ANSWER.md`

Deliverable	`~/linux-challenges/challenge7/ANSWER.md` (written explanation + commands)
Needs	Analysis challenge - the commands run on any Linux shell.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

• You must correctly identify the loopback-binding issue as the root cause from the `ss` output alone, and explain WHY it causes exactly this symptom.
• You must name at least 2 additional layers (beyond app binding) that could independently block connectivity even after this fix.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 7

```
$ ./linux_challenge.sh 7
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 7 of 20
================================================================================
 INCIDENT   INC0041939                                  PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : billing-api on port 8080 returns "Connection refused" for
                      every other server
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge7/
Read the brief:  cat ~/linux-challenges/challenge7/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge7/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge7/
ANSWER.md  PROBLEM.md  SYMPTOM_DATA.md
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge7 (on the billing server)

```
$ ss -tlnp | grep 8080
LISTEN 0 128 127.0.0.1:8080 0.0.0.0:* users:(("python3",pid=4471,fd=6))
# 127.0.0.1 = LOOPBACK. The packet never leaves the machine, so no other host can ever connect.
# The port is open - but only for processes on THIS box.

# Fix = application config: bind to 0.0.0.0 (all interfaces) or 10.0.1.55 (this interface), then restart
$ sudo systemctl restart billing-api
$ ss -tlnp | grep 8080
LISTEN 0 128 0.0.0.0:8080 0.0.0.0:* users:(("python3",pid=5120,fd=6))

# ---- still failing? test from the OTHER server and read the error type ----
$ nc -zv 10.0.1.55 8080
Connection to 10.0.1.55 8080 port [tcp/http-alt] succeeded!          # path is open end to end
# or one of these two failures:
nc: connect to 10.0.1.55 port 8080 (tcp) failed: Connection refused   # reached the host, nothing accepting
nc: connect to 10.0.1.55 port 8080 (tcp) failed: Connection timed out # packets silently DROPPED en route
$ traceroute -n 10.0.1.55                          # where does the path stop?

# ---- next layers to check ----
$ sudo firewall-cmd --list-all                     # host firewall on the billing server (Challenge 8)
# + cloud security group / network ACL between the two servers (outside the host)
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 7

```
# ss, ip, ping and traceroute work the same; the packages and the firewall front-end differ.
$ sudo dnf install -y nmap-ncat traceroute         # RHEL-family: nc comes from nmap-ncat
$ sudo apt install -y netcat-openbsd traceroute    # Ubuntu / Debian
$ sudo zypper install -y netcat-openbsd traceroute # SUSE
$ nc -zv 10.0.1.55 8080                            # same flags on all three
# Host firewall layer - ask whichever tool the distro uses:
$ sudo firewall-cmd --list-all                     # RHEL / Rocky / Alma / SUSE (firewalld)
$ sudo ufw status verbose                          # Ubuntu / Debian ("inactive" = no host firewall)
$ sudo nft list ruleset                            # any distro: what the kernel is really enforcing
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Solved (Permanently)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	06-Oct-2026 16:08
CLOSE NOTES	billing-api was bound to `127.0.0.1:8080`; rebound to `0.0.0.0:8080`, restarted and confirmed with `ss -tlnp` and `nc -zv` from a second host.		

KEY TAKEAWAYS

- Binding to `127.0.0.1` is a textbook mistake: many frameworks default to loopback for local-dev safety, and the default quietly reaches production.
- `ss`/`ip` are **diagnostic** tools - they show the problem but cannot change an app's bind address.
- "Listening correctly" only proves the **local** half. End-to-end reachability also depends on the **host firewall** and the **network security group / ACL**.
- **Refused** usually means the host answered but nothing accepted the connection; **timed out** usually means a firewall dropped it.
- `ss` reads kernel socket data via netlink and stays fast with huge connection counts; `netstat` parses `/proc/net/*` and is legacy.
- **Distro note:** `ss`, `ip`, `nc -zv` and `traceroute` behave the same everywhere; check the host firewall with `firewall-cmd` (RHEL / SUSE) or `ufw` (Ubuntu / Debian).

CHALLENGE

08

Firewall Management - firewalld / iptables Least-Privilege Rules

firewall-cmd | rich rules | --permanent | --reload | CIDR /16 /32

SERVICE REQUEST (RITM)

RITM0010549

PRIORITY

P3 - Moderate

SHORT DESCRIPTION	**Firewall rule request: 443/tcp from 10.0.0.0/16 and 8080/tcp from monitoring host 10.0.5.20 only**		
STATE	Work in Progress	OPENED	06-Oct-2026 13:55
REQUESTED FOR	Isabella Moreau (Application Team)	REQUESTED BY	Isabella Moreau (Application Team)
CATALOG ITEM	Firewall Rule Request	REQUEST	REQ0031087
APPROVAL	Approved (Security)	CATALOG TASK	SCTASK0010636
CATEGORY	Network Security	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03 (firewalld)	TARGET FULFILMENT	3 business days
ASSIGNMENT GROUP	Security Operations	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

This server's firewall (see ~/linux-challenges/challenge8/CURRENT_FIREWALL_STATE.md for the CURRENT `firewall-cmd --list-all` output) currently only allows SSH - the app team needs 2 NEW rules added, each with DIFFERENT, deliberately narrow scopes (see the requirement at the bottom of that file).

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Open 443/tcp to `10.0.0.0/16` with a **rich rule** - and explain why not plain `--add-port=443/tcp`.
2. Open 8080/tcp to exactly `10.0.5.20/32` - explain what `/32` means.
3. Explain `--permanent` vs runtime and the **permanent + reload** workflow.
4. Conceptually compare firewalld zones/rich rules with raw `iptables` chain ordering.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 8`
Workspace	`~/linux-challenges/challenge8/`

Files created	`~/linux-challenges/challenge8/PROBLEM.md`
	`~/linux-challenges/challenge8/CURRENT_FIREWALL_STATE.md`
	`~/linux-challenges/challenge8/firewall_commands.txt`

Deliverable	`~/linux-challenges/challenge8/firewall_commands.txt`
Needs	root + firewalld (RHEL / SUSE) or ufw (Ubuntu / Debian).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- Your port 443 rule must be scoped to the 10.0.0.0/16 CIDR specifically, not opened globally.
- Your port 8080 rule must be scoped to exactly 10.0.5.20/32.
- You must correctly explain the --permanent + --reload workflow.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 8

```
$ ./linux_challenge.sh 8
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 8 of 20
================================================================================
 SERVICE REQUEST (RITM)   RITM0010549                PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Firewall rule request: 443/tcp from 10.0.0.0/16 and
                      8080/tcp from monitoring host 10.0.5.20 only
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge8/
Read the brief:  cat ~/linux-challenges/challenge8/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge8/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge8/
CURRENT_FIREWALL_STATE.md  PROBLEM.md  firewall_commands.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge8

```
$ sudo firewall-cmd --permanent --zone=public --add-rich-rule=\
> 'rule family="ipv4" source address="10.0.0.0/16" port port="443" protocol="tcp" accept'
success
$ sudo firewall-cmd --permanent --zone=public --add-rich-rule=\
> 'rule family="ipv4" source address="10.0.5.20/32" port port="8080" protocol="tcp" accept'
success
$ sudo firewall-cmd --reload                      # load the permanent config into the running firewall
success

$ sudo firewall-cmd --list-rich-rules             # verify
rule family="ipv4" source address="10.0.0.0/16" port port="443" protocol="tcp" accept
rule family="ipv4" source address="10.0.5.20/32" port port="8080" protocol="tcp" accept
$ sudo firewall-cmd --list-ports                  # must stay EMPTY - nothing opened globally
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 8 (ufw, then raw iptables)

```
# Ubuntu / Debian use ufw instead of firewalld. Allow SSH FIRST or enabling ufw can cut your session.
$ sudo ufw default deny incoming                  # the default, stated explicitly
$ sudo ufw allow 22/tcp
$ sudo ufw allow from 10.0.0.0/16 to any port 443 proto tcp
Rule added
$ sudo ufw allow from 10.0.5.20 to any port 8080 proto tcp     # a bare IP means exactly one host (/32)
Rule added
$ sudo ufw enable                                 # ufw ships INACTIVE on Ubuntu
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
$ sudo ufw status numbered                        # representative output
Status: active

     To                         Action      From
     --                         ------      ----
[ 1] 22/tcp                     ALLOW IN    Anywhere
[ 2] 443/tcp                    ALLOW IN    10.0.0.0/16
[ 3] 8080/tcp                   ALLOW IN    10.0.5.20
# ufw rules apply IMMEDIATELY and survive reboots - there is no --permanent / --reload pair.
# ufw evaluates rules top to bottom and the first match wins, so order matters here too.

# Raw iptables (any distro) - ORDER MATTERS: the ACCEPT rules must come BEFORE the DROP.
```

```
Ubuntu / Debian equivalent - challenge 8 (ufw, then raw iptables)
$ sudo iptables -A INPUT -p tcp -s 10.0.0.0/16 --dport 443 -j ACCEPT
$ sudo iptables -A INPUT -p tcp -s 10.0.5.20/32 --dport 8080 -j ACCEPT
$ sudo iptables -A INPUT -p tcp -m multiport --dports 443,8080 -j DROP    # everyone else on those ports
$ sudo iptables -S INPUT
# make it persistent:  Ubuntu / Debian  sudo netfilter-persistent save     (package iptables-persistent)
#                      RHEL-family      sudo service iptables save         (package iptables-services)
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Request fulfilled
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	06-Oct-2026 15:48
CLOSE NOTES	Two rich rules added with `--permanent`: 443/tcp from `10.0.0.0/16` and 8080/tcp from `10.0.5.20/32`; `--reload` run and rules confirmed with `--list-rich-rules`.		

KEY TAKEAWAYS

- `--add-port=443/tcp` opens the port to **any** source reaching that interface. The rich rule adds a source CIDR, so even if the host becomes reachable from outside, the firewall still rejects it.
- `/32` = exactly one IPv4 address. `/24` would be a 256-address block. The requirement was literally one monitoring server.
- Without `--permanent` a rule is live but **lost on reload/reboot**. With `--permanent` it is saved but **not live until** `--reload`. Standard flow: add permanent, then reload.
- **iptables**: ordered rule lists - an earlier broad ALLOW can hide a later specific DENY. **firewalld**: zone-based abstraction with rich rules; it generates and orders the underlying nftables/iptables rules for you.
- **Distro note:** Ubuntu / Debian use `ufw`: rules apply immediately and persist (no `--permanent` / `--reload`), the firewall ships inactive, and SSH must be allowed before `ufw enable`. SUSE uses firewalld like RHEL.

CHALLENGE
09

Package Management - Repos, Dependency Conflicts & Pinning

yum / dnf | downgrade | versionlock | apt-mark hold | containers

INCIDENT
INC0041960

PRIORITY
P2 - High

SHORT DESCRIPTION	**Critical app crash-looping since fleet-wide yum update upgraded libfoo 1.4.2 to 1.6.0**		
STATE	In Progress	OPENED	06-Oct-2026 14:50
CALLER	Marcus Hale (Application Support)	CONTACT TYPE	Email
IMPACT	2 - Medium	URGENCY	1 - High
CATEGORY	Software / Packages	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03	TARGET RESOLUTION	8 hours
ASSIGNMENT GROUP	Linux Platform Ops	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

See ~/linux-challenges/challenge9/INCIDENT_CONTEXT.md - a routine, fleet-wide package update silently broke a critical app by upgrading a dependency out from under it, and now a SEPARATE new tool needs the NEWER version that's currently causing the incident - a real version conflict, not just a one-sided "downgrade and done" fix. (`libfoo` is a fictional package used for this challenge - the commands are the point.)

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Show installed and available versions of `libfoo`.
2. **Downgrade** to `1.4.2` - and what if the repo no longer serves it?
3. **Pin** `libfoo` so future updates never touch it, without freezing every other package.
4. Resolve the conflict with the new tool that needs 1.6.0+.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 9`
Workspace	`~/linux-challenges/challenge9/`

Files created	`~/linux-challenges/challenge9/PROBLEM.md`
	`~/linux-challenges/challenge9/INCIDENT_CONTEXT.md`
	`~/linux-challenges/challenge9/package_commands.txt`

Deliverable	`~/linux-challenges/challenge9/package_commands.txt`
Needs	root on any distro - yum / dnf, apt and zypper variants are all shown.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- You must give a real, correct downgrade command AND a real pinning command for whichever package manager family you choose, and stay consistent within that family throughout your answer.
- You must correctly identify containerization (or fixing the app) as the real resolution path for the underlying version conflict, not just "pin it and hope."

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 9

```
$ ./linux_challenge.sh 9
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 9 of 20
================================================================================
 INCIDENT   INC0041960                                  PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : Critical app crash-looping since fleet-wide yum update
                      upgraded libfoo 1.4.2 to 1.6.0
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge9/
Read the brief:  cat ~/linux-challenges/challenge9/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge9/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge9/
INCIDENT_CONTEXT.md  PROBLEM.md  package_commands.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge9 (RHEL / CentOS family)

```
$ yum list --showduplicates libfoo
Installed Packages
libfoo.x86_64     1.6.0-1.el9     @updates
Available Packages
libfoo.x86_64     1.4.2-1.el9     baseos
libfoo.x86_64     1.6.0-1.el9     updates

$ sudo yum downgrade -y libfoo-1.4.2              # back to the known-good version
# repo already moved on? use an internal artifact store / vault repo, then:
$ sudo yum downgrade ./libfoo-1.4.2-1.el9.x86_64.rpm

# ---- pin it: only THIS package is frozen, everything else keeps updating ----
$ sudo yum install -y yum-plugin-versionlock       # RHEL 8/9: python3-dnf-plugin-versionlock
$ sudo yum versionlock add libfoo-1.4.2
$ yum versionlock list
libfoo-0:1.4.2-1.el9.*
# Debian / Ubuntu equivalent:   sudo apt-mark hold libfoo

# ---- the new tool that needs libfoo >= 1.6.0 ----
$ docker run --rm newtool:latest --version         # option (a): container with its own libs
# option (b): patch/upgrade the crashing app to work with 1.6.0, then drop the pin
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian, RHEL (dnf) and SUSE equivalents - challenge 9

```
# ---- Ubuntu / Debian (apt) ----
$ apt-cache policy libfoo                          # installed / candidate / all versions (or apt list -a)
$ sudo apt install libfoo=1.4.2-1 --allow-downgrades     # back to the known-good version
# repo no longer serves it? install the .deb you kept:   sudo dpkg -i ./libfoo_1.4.2-1_amd64.deb
$ sudo apt-mark hold libfoo                        # pin: apt upgrade now skips it
libfoo set on hold.
$ apt-mark showhold
libfoo
# undo: sudo apt-mark unhold libfoo.  Companion packages (libfoo-dev ...) must be the SAME version.

# ---- RHEL 8/9, Rocky, Alma, Fedora (dnf - "yum" is just an alias) ----
$ dnf list --showduplicates libfoo
$ sudo dnf downgrade libfoo-1.4.2-1.el9
$ sudo dnf install -y 'dnf-command(versionlock)'   # installs python3-dnf-plugin-versionlock
$ sudo dnf versionlock add libfoo-1.4.2
```

```
Ubuntu / Debian, RHEL (dnf) and SUSE equivalents - challenge 9
$ dnf versionlock list

# ---- SUSE / openSUSE (zypper) ----
$ zypper search -s libfoo                          # -s = show every available version
$ sudo zypper install --oldpackage libfoo=1.4.2    # allow replacing a newer package with an older one
$ sudo zypper addlock libfoo                       # pin;  zypper locks = list;  removelock = undo

# ---- the container workaround ----
$ docker run --rm newtool:latest --version         # Ubuntu / Debian (after: sudo apt install docker.io)
$ podman run --rm newtool:latest --version         # RHEL 8/9 ships podman instead of docker
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Solved (Work Around)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	06-Oct-2026 16:33
CLOSE NOTES	`libfoo` downgraded to 1.4.2 and pinned with `yum versionlock`; the tool needing 1.6.0+ isolated in a container until the app is fixed.		

KEY TAKEAWAYS

- `yum downgrade` targets one exact version; "remove and reinstall" would just fetch the latest again.
- Older versions do not stay in a repo forever - you may need a vault repo or the exact RPM from an internal artifact store.
- **versionlock** protects one package; "never run updates on this server" would leave the whole system unpatched.
- Two versions of one system library cannot coexist via the package manager. **Container** = quick workaround. **Fixing the app for 1.6.0** = the durable fix that removes the pin entirely.
- **Distro note:** the pin is `apt-mark hold` on Ubuntu / Debian, `dnf versionlock` on RHEL-family and `zypper addlock` on SUSE. APT keeps very few old versions, so keep the `.deb` (or a mirror) of anything you may need to roll back to.

CHALLENGE
10

Shell Scripting Fundamentals - A Real Production-Style Script

bash | set -euo pipefail | functions | exit codes | process substitution

PROBLEM RECORD
PRB0040095

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**disk_check.sh breaks on mount points with spaces and never returns a failing exit code**		
STATE	Root Cause Analysis	OPENED	06-Oct-2026 17:19
CATEGORY	Monitoring / Scripting	BUSINESS SERVICE	Infrastructure Monitoring
CONFIGURATION ITEM	disk_check.sh	RELATED INCIDENTS	3 incidents
KNOWN ERROR	Yes - workaround documented	TARGET RESOLUTION	3 business days
ASSIGNMENT GROUP	IT Operations - L2	ASSIGNED TO	You (DevOps Engineer)
PROBLEM STATEMENT	The disk threshold script mis-parses mount points containing spaces, has a hard-coded threshold, runs its loop in a subshell and always exits 0, so cron and monitoring cannot rely on it.		

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

The starter script (~/linux-challenges/challenge10/disk_check.sh.starter) is meant to alert when any mounted filesystem crosses a disk-usage threshold - it mostly "works" in casual testing, but has several real, production- grade bugs that a code review caught: it silently produces WRONG or CRASHING output whenever a mount point contains a SPACE in its name (common with some NFS/network mounts), it has no configurable threshold via command-line argument, it doesn't exit with a meaningful non-zero code when something IS over threshold (so it can't be reliably used inside a monitoring/cron pipeline that checks exit codes), and it doesn't handle the `while read` loop running in a SUBSHELL correctly if you were to try to accumulate a count of warnings across iterations.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Accept an optional threshold: `./disk_check.sh 85` (default 90) using `${1:-90}`.
2. Handle **mount points with spaces** - quote everything, use `df -P`.
3. Add `set -euo pipefail` and explain `-e`, `-u`, `-o pipefail`.
4. Exit `1` if anything is over threshold, else `0` - and fix the **subshell** gotcha.
5. Move the per-line logic into a **named function**.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 10`
Workspace	`~/linux-challenges/challenge10/`

Files created	`~/linux-challenges/challenge10/PROBLEM.md`
	`~/linux-challenges/challenge10/disk_check.sh.starter`
	`~/linux-challenges/challenge10/disk_check.sh`

Deliverable	`~/linux-challenges/challenge10/disk_check.sh`
Needs	Runs anywhere (any Linux shell).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- The script must correctly handle a mount point with a space in its name without crashing or misparsing.
- The script must exit 1 if any filesystem is over threshold, exit 0 otherwise - verified by testing both conditions.
- `set -euo pipefail` must be present at the top.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 10

```
$ ./linux_challenge.sh 10
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 10 of 20
================================================================================
 PROBLEM RECORD   PRB0040095                          PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : disk_check.sh breaks on mount points with spaces and
                      never returns a failing exit code
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge10/
Read the brief:  cat ~/linux-challenges/challenge10/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge10/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge10/
PROBLEM.md  disk_check.sh  disk_check.sh.starter
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge10

```
$ cd ~/linux-challenges/challenge10
# a fake df lets us test a mount point WITH SPACES without touching the real system
$ mkdir -p fake && cat > fake/df <<'EOF'
#!/bin/bash
cat <<'DATA'
Filesystem     1024-blocks    Used Available Capacity Mounted on
/dev/sda1         20000000 19000000   1000000      95% /
nas:/export        1000000  920000     80000      92% /mnt/my nfs share
DATA
EOF
$ chmod +x fake/df

# ---- reproduce the bugs in the starter ----
$ cp disk_check.sh.starter starter.sh && chmod +x starter.sh
$ PATH=$PWD/fake:$PATH ./starter.sh; echo "exit=$?"
WARNING: / is at 95%
WARNING: /mnt/my is at 92%
Disk check complete.
exit=0
# bug 1: "/mnt/my nfs share" was cut at the first space   bug 2: breaches found, yet the exit code is 0

# ---- the fixed script ----
$ cat > disk_check.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

THRESHOLD="${1:-90}"                 # optional argument, default 90
OVER_THRESHOLD=0

check_line() {                       # one function = one job
  local usage="$1" mount="$2"
  if [ "$usage" -gt "$THRESHOLD" ]; then
    echo "WARNING: $mount is at ${usage}% - over threshold!"
    return 1
  fi
  return 0
}

# the LAST variable (mount) receives the rest of the line, so spaces survive
while read -r _fs _size _used _avail pct mount; do
  usage="${pct%\%}"
  if ! check_line "$usage" "$mount"; then
    OVER_THRESHOLD=1
```

bash - challenge10

```
  fi
done < <(df -P | tail -n +2)          # process substitution keeps the loop in THIS shell

echo "Disk check complete."
exit "$OVER_THRESHOLD"
EOF
$ chmod +x disk_check.sh

# ---- test 1: default threshold 90 -> both mounts breach, including the one with spaces ----
$ PATH=$PWD/fake:$PATH ./disk_check.sh; echo "exit=$?"
WARNING: / is at 95% - over threshold!
WARNING: /mnt/my nfs share is at 92% - over threshold!
Disk check complete.
exit=1

# ---- test 2: threshold 99 -> nothing breaches ----
$ PATH=$PWD/fake:$PATH ./disk_check.sh 99; echo "exit=$?"
Disk check complete.
exit=0

# ---- test 3: threshold 93 -> only / (95%) breaches ----
$ PATH=$PWD/fake:$PATH ./disk_check.sh 93; echo "exit=$?"
WARNING: / is at 95% - over threshold!
Disk check complete.
exit=1
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 10

```
# Gotcha 1: /bin/sh is dash on Ubuntu / Debian (bash on RHEL). This script needs bash features.
$ readlink -f /bin/sh                           # RHEL: /usr/bin/bash   Ubuntu / Debian: /usr/bin/dash
/usr/bin/dash
$ bash ./disk_check.sh 85                       # correct on every distro (or ./disk_check.sh)
$ sh ./disk_check.sh 85                         # WRONG on Ubuntu / Debian: dash rejects it
# the shebang "#!/usr/bin/env bash" makes ./disk_check.sh safe - never run "sh disk_check.sh"

# Gotcha 2: snap packages are read-only squashfs mounts that always report 100% used. On Ubuntu the default
# threshold would warn about /snap/... on EVERY run and exit 1. Filter those filesystems out of df:
#     done < <(df -P -x tmpfs -x squashfs -x devtmpfs | tail -n +2)      # the one changed line
$ bash ./disk_check.sh 90; echo "exit=$?"       # with the filter, only REAL filesystems are checked
Disk check complete.
exit=0
$ bash ./disk_check.sh 1; echo "exit=$?"        # force a breach to prove the exit code still works
WARNING: / is at 3% - over threshold!
Disk check complete.
exit=1
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Root cause identified; fix applied
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	06-Oct-2026 18:58
CLOSE NOTES	`disk_check.sh` rewritten: threshold argument (`${1:-90}`), `read -r` with `df -P` for spaces, process substitution, `set -euo pipefail`, exit 1 on breach.		

KEY TAKEAWAYS

- **Unquoted** `$line` / `$mount` get re-split on whitespace - the classic bug that broke the starter on `/mnt/my nfs share`.
- `-e` stops on a failing command, `-u` fails on unset variables (catches typos like `$THRESHHOLD`), `-o pipefail` makes a pipeline fail if **any** stage fails, not just the last.
- `df | while read ...` runs the loop in a **subshell**; variables set inside vanish. `done < <(df -P ...)` keeps it in the parent shell.
- `read -r ... pct mount` lets the **last** variable absorb the remainder of the line. `awk '{print $6}'` or `cut -d" " -f6` cut the path at its first space (and `df` pads columns with several spaces).
- Named functions make logic readable, reusable and testable on their own (`check_line 95 /data`).
- **Distro note:** `/bin/sh` is **dash** on Ubuntu / Debian but **bash** on RHEL, so run the script as `./disk_check.sh` or `bash disk_check.sh`, never `sh disk_check.sh`. Ubuntu also mounts every snap as a read-only squashfs at 100% used - filter them with `df -P -x tmpfs -x squashfs -x devtmpfs` or the script will warn on every run.

CHALLENGE
11

Text Processing - grep, awk, sed & the Unix Pipeline

grep | awk | sed | sort | uniq -c | one-liners

INCIDENT
INC0041978

PRIORITY
P2 - High

SHORT DESCRIPTION	**Customers report failed billing charges - HTTP 500 on /api/billing/charge**		
STATE	In Progress	OPENED	07-Oct-2026 08:42
CALLER	Chloe Dubois (Customer Support)	CONTACT TYPE	Email
IMPACT	2 - Medium	URGENCY	1 - High
CATEGORY	Application / Errors	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	web-srv-02 (access.log)	TARGET RESOLUTION	8 hours
ASSIGNMENT GROUP	IT Operations - L2	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

Customers are reporting failed billing charges. You have raw web server access logs at ~/linux-challenges/challenge11/access.log (a small sample - imagine this is really millions of lines) and need to answer several questions FAST, using only command-line text processing tools - no scripts, no external log platform, just the Unix pipeline, as if you were live-debugging over an SSH session during an incident.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Count how many requests returned **500**.
2. List the **unique IPs** that got a 500 on `/api/billing/charge`, with counts, most frequent first.
3. Extract **METHOD + PATH** from every line, show each unique pair once with its count (descending).
4. Use `sed` to **redact the source IP** on every line.
5. Why `awk` rather than `cut` for question 3?

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 11`
Workspace	`~/linux-challenges/challenge11/`

Files created	`~/linux-challenges/challenge11/PROBLEM.md`
	`~/linux-challenges/challenge11/access.log`
	`~/linux-challenges/challenge11/commands.txt`

Deliverable	`~/linux-challenges/challenge11/commands.txt`
Needs	Runs anywhere (any Linux shell).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- Each command must be a real, runnable one-liner producing the exact requested output against the sample log file.
- Your IP-frequency list for question 2 must be correctly sorted by count, descending.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 11

```
$ ./linux_challenge.sh 11
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 11 of 20
================================================================================
 INCIDENT   INC0041978                                    PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : Customers report failed billing charges - HTTP 500 on
                      /api/billing/charge
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge11/
Read the brief:  cat ~/linux-challenges/challenge11/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge11/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge11/
PROBLEM.md  access.log  commands.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge11

```
$ cd ~/linux-challenges/challenge11
$ head -3 access.log
10.0.1.5 - - [14/Sep/2026:08:01:12] "GET /api/health HTTP/1.1" 200 45
10.0.2.9 - - [14/Sep/2026:08:01:15] "POST /api/billing/charge HTTP/1.1" 500 120
10.0.1.5 - - [14/Sep/2026:08:01:20] "GET /api/health HTTP/1.1" 200 45

# 1) total 500s
$ grep -c '" 500 ' access.log
4

# 2) unique IPs with a 500 on the billing endpoint, most frequent first
$ grep '/api/billing/charge' access.log | grep '" 500 ' | awk '{print $1}' | sort | uniq -c | sort -rn
      4 10.0.2.9

# 3) unique METHOD + PATH with counts, descending
$ awk -F'"' '{print $2}' access.log | awk '{print $1, $2}' | sort | uniq -c | sort -rn
      5 POST /api/billing/charge
      3 GET /api/health
      1 GET /api/users/42
      1 GET /api/orders

# 4) redact the source IP (first field) without touching the rest of the line
$ sed -E 's/^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+/REDACTED/' access.log | head -2
REDACTED - - [14/Sep/2026:08:01:12] "GET /api/health HTTP/1.1" 200 45
REDACTED - - [14/Sep/2026:08:01:15] "POST /api/billing/charge HTTP/1.1" 500 120
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 11

```
# grep, sed and sort | uniq -c are identical. Two real differences: which awk you get, and where logs live.
$ readlink -f "$(command -v awk)"                  # Ubuntu / Debian default: mawk     RHEL: gawk
/usr/bin/mawk
# the awk one-liners in this challenge use only features both have, so they run unchanged
$ awk -F'"' '{print $2}' access.log | awk '{print $1, $2}' | sort | uniq -c | sort -rn | head -2
      5 POST /api/billing/charge
      3 GET /api/health

# Real web-server logs (path depends on the server and the distro):
$ sudo grep -c '" 500 ' /var/log/nginx/access.log          # nginx - Ubuntu / Debian and RHEL
$ sudo grep -c '" 500 ' /var/log/apache2/access.log        # Apache on Ubuntu / Debian
```

```
Ubuntu / Debian equivalent - challenge 11
$ sudo grep -c '" 500 ' /var/log/httpd/access_log          # Apache on RHEL-family
# logrotate gzips the older files - zcat -f reads plain AND .gz files together:
$ sudo zcat -f /var/log/nginx/access.log* | grep -c '" 500 '        # e.g. today's file + one rotated .gz
8
$ sudo zgrep -c '" 500 ' /var/log/nginx/access.log.*.gz             # count per rotated file
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Solved (Permanently)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	07-Oct-2026 12:51
CLOSE NOTES	Log analysis: 4 HTTP 500 responses, all on `/api/billing/charge` from a single client `10.0.2.9`. One-liners saved for reuse.		

KEY TAKEAWAYS

- The story the data tells: **one client, `10.0.2.9`, caused all 4 failures** on `/api/billing/charge` - the only IP with a 500 there. (A fifth charge, from `10.0.5.1` , succeeded with 200.)
- `sort | uniq -c | sort -rn` is the universal "count and rank" idiom - `uniq` only collapses *adjacent* duplicates, so `sort` must come first.
- `awk -F'"'` splits on the **quote characters**, so the whole quoted request (`GET /path HTTP/1.1`) becomes one field even though it contains spaces. `cut -d" "` would grab only one word.
- Anchoring the `sed` pattern with `^` replaces only the leading IP - never an IP-looking string elsewhere in the line.
- **Distro note:** Ubuntu / Debian ship **mawk** as `awk` (RHEL ships gawk); these one-liners run on both. Log paths differ: `/var/log/nginx/`, `/var/log/apache2/access.log` on Ubuntu / Debian vs `/var/log/httpd/access_log` on RHEL - and `zcat -f` reads rotated `.gz` logs together with the current one.

CHALLENGE
12

Cron & Scheduled Tasks - crontab and systemd Timers

crontab | at | systemd timer + service | OnCalendar | Persistent=true

SERVICE REQUEST (RITM)
RITM0010568

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**Schedule nightly DB backup on weekdays 02:30, a one-time cleanup at 23:45, and start-failure visibility**		
STATE	Work in Progress	OPENED	07-Oct-2026 09:44
REQUESTED FOR	Nathan Brooks (DBA Team)	REQUESTED BY	Nathan Brooks (DBA Team)
CATALOG ITEM	Job Scheduling Request	REQUEST	REQ0031131
APPROVAL	Approved (Service Owner)	CATALOG TASK	SCTASK0010679
CATEGORY	Job Scheduling	BUSINESS SERVICE	Billing Database
CONFIGURATION ITEM	db-srv-01 (backup_db.sh)	TARGET FULFILMENT	3 business days
ASSIGNMENT GROUP	IT Operations - L2	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

See ~/linux-challenges/challenge12/REQUIREMENTS.md for 3 distinct scheduling requirements around ~/linux-challenges/challenge12/backup_db.sh and a separate one-off task.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Write the exact **crontab line** and explain the 5 fields.
2. Use `at` for the one-time job - why not cron?
3. Why does plain cron give **no visibility** into "did it start"? What is the systemd alternative?
4. Write the `.timer` + `.service` pair for the same weekday 2:30 AM schedule.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 12`
Workspace	`~/linux-challenges/challenge12/`

Files created	`~/linux-challenges/challenge12/PROBLEM.md`
	`~/linux-challenges/challenge12/backup_db.sh`
	`~/linux-challenges/challenge12/cleanup_once.sh`
	`~/linux-challenges/challenge12/REQUIREMENTS.md`
	`~/linux-challenges/challenge12/crontab_entry.txt`
	`~/linux-challenges/challenge12/billing-backup.timer`
	`~/linux-challenges/challenge12/billing-backup.service`

Deliverable	`~/linux-challenges/challenge12/crontab_entry.txt`
	`~/linux-challenges/challenge12/billing-backup.timer`
	`~/linux-challenges/challenge12/billing-backup.service`

Needs	cron, `at` and systemd (replace `student` with your username in paths). On RHEL the cron service is `crond`.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- Your crontab line must correctly express "2:30 AM, Monday-Friday only" using proper 5-field cron syntax.
- Your `at` command must be genuinely one-time, not a recurring cron entry disguised as one-time.
- Your systemd timer's OnCalendar= expression must correctly express the same weekday/time schedule.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 12

```
$ ./linux_challenge.sh 12
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 12 of 20
================================================================================
 SERVICE REQUEST (RITM)   RITM0010568                 PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Schedule nightly DB backup on weekdays 02:30, a one-time
                      cleanup at 23:45, and start-failure visibility
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge12/
Read the brief:  cat ~/linux-challenges/challenge12/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge12/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge12/
PROBLEM.md  REQUIREMENTS.md  backup_db.sh  billing-backup.service  billing-backup.timer
cleanup_once.sh  crontab_entry.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge12

```
# ---- 1) cron: minute hour day-of-month month day-of-week ----
$ crontab -e
30 2 * * 1-5 /home/student/linux-challenges/challenge12/backup_db.sh
$ crontab -l
30 2 * * 1-5 /home/student/linux-challenges/challenge12/backup_db.sh

# ---- 2) one-time job with at (no recurrence, nothing to clean up) ----
$ echo "/home/student/linux-challenges/challenge12/cleanup_once.sh" | at 23:45
warning: commands will be executed using /bin/sh
job 4 at Sun Oct 11 23:45:00 2026
$ atq
4       Sun Oct 11 23:45:00 2026 a student

# ---- 3) systemd timer + service ----
$ sudo tee /etc/systemd/system/billing-backup.service <<'EOF'
[Unit]
Description=Nightly billing DB backup

[Service]
Type=oneshot
ExecStart=/home/student/linux-challenges/challenge12/backup_db.sh
EOF
$ sudo tee /etc/systemd/system/billing-backup.timer <<'EOF'
[Unit]
Description=Run billing-backup.service weekdays at 2:30 AM

[Timer]
OnCalendar=Mon..Fri 02:30:00
Persistent=true

[Install]
WantedBy=timers.target
EOF
$ systemd-analyze calendar 'Mon..Fri 02:30:00'     # validate the schedule expression
  Original form: Mon..Fri 02:30:00
Normalized form: Mon..Fri *-*-* 02:30:00
    Next elapse: Mon 2026-10-12 02:30:00 UTC
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now billing-backup.timer
$ systemctl list-timers billing-backup.timer       # last run / next run, at a glance
$ journalctl -u billing-backup.service             # full history incl. "failed to start" entries
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 12

```
# crontab syntax and systemd timers are identical everywhere. The service name and the install step differ.
$ systemctl status crond                          # RHEL / Rocky / Alma
$ systemctl status cron                           # Ubuntu / Debian / SUSE
$ journalctl -u cron --since today                # cron's log (RHEL: -u crond, or /var/log/cron)
# "at" is often NOT installed by default - install it and start atd on any distro:
$ sudo dnf install -y at && sudo systemctl enable --now atd        # RHEL-family
$ sudo apt install -y at && sudo systemctl enable --now atd        # Ubuntu / Debian
$ sudo zypper install -y at && sudo systemctl enable --now atd     # SUSE
# OnCalendar=Mon..Fri 02:30:00 and Persistent=true are identical on every systemd distro.
```

RESOLUTION INFORMATION

STATE	Resolved	CLOSE CODE	Request fulfilled
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	07-Oct-2026 14:02
CLOSE NOTES	Crontab `30 2 * * 1-5`; one-time `at 23:45` job; `billing-backup.timer` + `.service` with `Persistent=true` for observability.		

KEY TAKEAWAYS

- Cron fields in order: **minute** (30), **hour** (2), **day-of-month** (`*`), **month** (`*`), **day-of-week** (`1-5` = Monday-Friday).
- `at` is built for one-off jobs. A "one-time" cron line that is never removed will quietly fire again at 23:45 **every night**. (`at` is not installed by default on RHEL: `dnf install at` and enable `atd`.)
- Plain cron has no "attempted to run" record: if the daemon or server is down at 2:30, nothing happens - no log, no error, no alert. A **timer** gives `list-timers`, `OnFailure=` hooks and a full `journalctl` history.
- `Persistent=true` runs a **missed** job as soon as the machine is back - cron has no equivalent.
- **Distro note:** the cron service is `crond` on RHEL but `cron` on Ubuntu / Debian / SUSE. `at` usually has to be installed and its `atd` service enabled on every distro.

CHALLENGE
13

Log Management - journalctl, /var/log & logrotate

logrotate | delaycompress | postrotate vs copytruncate | journalctl -u / -f / -p

INCIDENT
INC0041994

PRIORITY
P2 - High

SHORT DESCRIPTION	**/var/log filled the root filesystem overnight; logs for the incident window also needed**		
STATE	In Progress	OPENED	07-Oct-2026 10:54
CALLER	Monitoring alert: / filesystem > 90%	CONTACT TYPE	Monitoring alert
IMPACT	2 - Medium	URGENCY	2 - Medium
CATEGORY	Storage / Logging	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03 (/var/log)	TARGET RESOLUTION	8 hours
ASSIGNMENT GROUP	Linux Platform Ops	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

See ~/linux-challenges/challenge13/INCIDENT_CONTEXT.md - two related but distinct log-management problems.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Write a **logrotate** config: daily, keep 14, compress with `delaycompress`, and `postrotate` or `copytruncate` (justify).
2. Give four `journalctl` commands: unit logs, a time window, live follow, errors only.
3. Why does the output land in the journal, and what is the trade-off?

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 13`
Workspace	`~/linux-challenges/challenge13/`

Files created	`~/linux-challenges/challenge13/PROBLEM.md`
	`~/linux-challenges/challenge13/INCIDENT_CONTEXT.md`
	`~/linux-challenges/challenge13/var_log_sim/app-1.log ... app-5.log`
	`~/linux-challenges/challenge13/billing-app.logrotate`
	`~/linux-challenges/challenge13/journalctl_commands.txt`

Deliverable	`~/linux-challenges/challenge13/billing-app.logrotate` `~/linux-challenges/challenge13/journalctl_commands.txt`
Needs	root + logrotate + systemd.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- Your logrotate snippet must include daily rotation, a 14-day retention count, compression, and a correctly justified choice between copytruncate and postrotate/signal-based rotation.
- Your journalctl commands must correctly use `-u`, `--since`/ `--until`, `-f`, and `-p` respectively for each sub-question.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 13

```
$ ./linux_challenge.sh 13
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 13 of 20
================================================================================
 INCIDENT   INC0041994                                    PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : /var/log filled the root filesystem overnight; logs for
                      the incident window also needed
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge13/
Read the brief:  cat ~/linux-challenges/challenge13/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge13/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge13/
INCIDENT_CONTEXT.md  PROBLEM.md  billing-app.logrotate  journalctl_commands.txt  var_log_sim
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge13

```
$ sudo tee /etc/logrotate.d/billing-app <<'EOF'
/var/log/billing-app/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    postrotate
        systemctl reload billing-app > /dev/null 2>&1 || true
    endscript
}
EOF
# NOTE: reload only works if the app's unit defines ExecReload= (e.g. ExecReload=/bin/kill -HUP $MAINPID)
# and the app reopens its log file on SIGHUP - if it does not, use copytruncate instead of postrotate
$ sudo logrotate -d /etc/logrotate.d/billing-app   # dry-run / debug: shows what WOULD happen
$ sudo logrotate -f /etc/logrotate.d/billing-app   # force one rotation to prove it works

# ---- journalctl ----
$ journalctl -u billing-app                                  # (a) everything for this unit
$ journalctl -u billing-app --since "22:00" --until "22:30"   # (b) incident window
$ journalctl -u billing-app -f                               # (c) follow live (like tail -f)
$ journalctl -u billing-app -p err                           # (d) ERROR and more severe
# priorities: 0 emerg  1 alert  2 crit  3 err  4 warning  5 notice  6 info  7 debug
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 13

```
# journalctl and logrotate work identically on every distro. The plain-text logs live in different files:
$ sudo tail -f /var/log/messages                  # RHEL / Rocky / Alma: general system log
$ sudo tail -f /var/log/secure                    # RHEL-family: SSH / sudo / login events
$ sudo tail -f /var/log/syslog                    # Ubuntu / Debian: general system log
$ sudo tail -f /var/log/auth.log                  # Ubuntu / Debian: SSH / sudo / login events
# (if rsyslog is installed; Debian 12+ often has only the journal, so use journalctl)
# The same events, distro-independent - mind the SSH service name:
$ journalctl -u ssh --since "22:00"               # Ubuntu / Debian service is "ssh"
$ journalctl -u sshd --since "22:00"              # RHEL / SUSE service is "sshd"
# /etc/logrotate.d/ drop-ins, the directives and logrotate -d / -f are the same on all of them.
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Solved (Permanently)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	07-Oct-2026 14:45
CLOSE NOTES	logrotate policy in place (daily, rotate 14, compress, delaycompress, postrotate reload); incident window pulled with `journalctl --since/--until`.		

KEY TAKEAWAYS

- `delaycompress` leaves the newest rotated file uncompressed for one more cycle - a process may still hold a handle to it for a moment after rotation.
- `postrotate` + reload tells the app to **reopen** its log file: no data loss. `copytruncate` (copy then truncate in place) is the fallback for apps with no reload signal, but lines written between copy and truncate can be **lost**.
- systemd captures a service's stdout/stderr into the **journal by default** - no logging config needed. It is structured and filterable, but an engineer who only knows `grep /var/log/*.log` will not find it.
- `-p err` shows priority 3 and everything more severe (0-3).
- **Distro note:** system and auth logs are `/var/log/messages` + `/var/log/secure` on RHEL but `/var/log/syslog` + `/var/log/auth.log` on Ubuntu / Debian. `journalctl` and `logrotate` are the same everywhere.

CHALLENGE

14

SSH Configuration & Hardening - Key-Based Auth Done Right

ssh-keygen ed25519 | authorized_keys | sshd_config | sshd -t

CHANGE REQUEST

CHG0030734

PRIORITY

P3 - Moderate

SHORT DESCRIPTION	**Remediate SSH audit findings: root login, password auth, X11, retries and idle sessions**		
STATE	Implement	OPENED	07-Oct-2026 13:22
TYPE	Normal	RISK	Moderate
REQUESTED BY	Hannah Lindqvist (Security Compliance)	APPROVAL	Approved (CAB)
CATEGORY	Security Hardening	BUSINESS SERVICE	Linux Server Access
CONFIGURATION ITEM	app-srv-03 (sshd)	CHANGE TASK	CTASK0030984
PLANNED START	08-Oct-2026 22:00	PLANNED END	09-Oct-2026 00:00
ASSIGNMENT GROUP	Cloud & DevOps	ASSIGNED TO	You (DevOps Engineer)
JUSTIFICATION	Security audit flagged PermitRootLogin yes, PasswordAuthentication yes, X11Forwarding yes, MaxAuthTries 6 and ClientAliveInterval 0.		

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

A security audit of this server's SSH configuration (see ~/linux-challenges/challenge14/current_sshd_config.md) flagged every listed setting - this is functionally the SAME hardening exercise real teams face constantly, so treat it seriously and completely.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Generate an SSH key pair (algorithm choice: **ed25519** vs rsa).
2. Install the public key and explain the **700 / 600** permissions.
3. Rewrite **every** flagged line and justify each change.
4. Validate the config with `sshd -t` **before** restarting - and explain why.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 14`
Workspace	`~/linux-challenges/challenge14/`

Files created	`~/linux-challenges/challenge14/PROBLEM.md`
	`~/linux-challenges/challenge14/current_sshd_config.md`
	`~/linux-challenges/challenge14/hardened_sshd_config.md`

Deliverable	`~/linux-challenges/challenge14/hardened_sshd_config.md`
Needs	root + sshd on a VM (and a **second terminal** for the safety test). Ubuntu / Debian: the service is `ssh`.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- Your final sshd_config must reflect all the hardened settings listed above, correctly.
- You must correctly explain the "test key auth in a second session BEFORE disabling password auth" safety step.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 14

```
$ ./linux_challenge.sh 14
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 14 of 20
================================================================================
 CHANGE REQUEST   CHG0030734                        PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Remediate SSH audit findings: root login, password auth,
                      X11, retries and idle sessions
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge14/
Read the brief:  cat ~/linux-challenges/challenge14/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge14/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge14/
PROBLEM.md  current_sshd_config.md  hardened_sshd_config.md
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge14

```
# ---- on YOUR machine ----
$ ssh-keygen -t ed25519 -C "emma.h@company.com"
Generating public/private ed25519 key pair.
Your identification has been saved in /home/emma/.ssh/id_ed25519
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub emma.h@server
# manual equivalent, on the server:
$ mkdir -p ~/.ssh && cat id_ed25519.pub >> ~/.ssh/authorized_keys
$ chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys

# ---- SAFETY: prove key login works in a SECOND terminal, keep the first one open ----
$ ssh -i ~/.ssh/id_ed25519 -o PasswordAuthentication=no emma.h@server

# ---- harden /etc/ssh/sshd_config ----
$ sudo vi /etc/ssh/sshd_config
$ sudo grep -E '^(PermitRoot|PasswordAuth|X11|MaxAuth|ClientAlive)' /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
X11Forwarding no
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2

$ sudo sshd -t                                  # syntax check - no output means the config is valid
$ echo $?
0
$ sudo systemctl reload sshd                    # (service is called "ssh" on Debian/Ubuntu)
# RHEL 9 / cloud images: also check /etc/ssh/sshd_config.d/*.conf (a drop-in can override your setting)
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 14

```
# ssh-keygen, authorized_keys permissions and every sshd_config option are identical. The service differs:
$ sudo sshd -t                                      # validate - same everywhere
$ sudo systemctl reload ssh                         # unit is "ssh" on Ubuntu / Debian (RHEL / SUSE: sshd)
# Ubuntu 22.10+ starts sshd through a systemd socket (ssh.socket):
#   normal setting changes (PermitRootLogin, PasswordAuthentication ...)  -> reload ssh is enough
#   changing the PORT / ListenAddress needs:  systemctl daemon-reload && systemctl restart ssh.socket
# A drop-in file can silently override your edit (cloud images often ship one that re-enables passwords):
$ ls /etc/ssh/sshd_config.d/
50-cloud-init.conf
$ sudo sshd -T | grep -i -E 'passwordauthentication|permitrootlogin'     # the EFFECTIVE values
permitrootlogin no
passwordauthentication no
# If you ever move SSH to another port, open it in the host firewall first:  sudo ufw allow <port>/tcp
```

RESOLUTION INFORMATION

STATE	Closed - Successful	CLOSE CODE	Successful
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	09-Oct-2026 00:00
CLOSE NOTES	Key login proven first; `PermitRootLogin no`, `PasswordAuthentication no`, `X11Forwarding no`, `MaxAuthTries 3`, `ClientAliveInterval 300` applied; `sshd -t` clean before reload.		

KEY TAKEAWAYS

- **ed25519**: shorter keys, equal or better security, faster generation and verification than RSA.
- SSH **refuses** an `authorized_keys` file or `~/.ssh` directory that is group/world-writable - otherwise another user could append their own key and walk in.
- `PermitRootLogin no`: log in as a named user + `sudo` for an audit trail of **who** did what.
- `PasswordAuthentication no` removes the whole class of password guessing/stuffing attacks - **but only after** key login is proven in a separate session, otherwise you can lock everyone out.
- `MaxAuthTries 3` shrinks the brute-force window. `ClientAliveInterval 300` + `CountMax 2` drops dead sessions after ~10 minutes. `X11Forwarding no` removes needless attack surface.
- `sshd -t` validates syntax without touching the running service; a broken config restart can kill SSH for new connections.
- **Distro note:** the service is `ssh` on Ubuntu / Debian (`sshd` on RHEL / SUSE), and Ubuntu 22.10+ uses socket activation - port changes need `daemon-reload` + `restart ssh.socket`. On any distro, `sshd -T` shows the effective values after drop-ins in `/etc/ssh/sshd_config.d/`.

CHALLENGE
15

find & xargs - Safe, Efficient Batch File Operations

find -mtime -delete | find -perm | -print0 / xargs -0 | dry runs

CHANGE REQUEST
CHG0030764

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**Standard change: purge stale tmp uploads and remove world-writable permissions from app_data**		
STATE	Implement	OPENED	07-Oct-2026 15:43
TYPE	Standard	RISK	Low
REQUESTED BY	Hannah Lindqvist (Security Compliance)	APPROVAL	Pre-approved (Standard template)
CATEGORY	Security / Housekeeping	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03 (app_data)	CHANGE TASK	CTASK0031022
PLANNED START	08-Oct-2026 22:00	PLANNED END	09-Oct-2026 00:00
ASSIGNMENT GROUP	Linux Platform Ops	ASSIGNED TO	You (DevOps Engineer)
JUSTIFICATION	A security scan found world-writable config files (`secrets.conf` and `legacy settings.conf`); `tmp_uploads/` also holds `.tmp` files older than 7 days.		

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

Two cleanup/audit tasks on ~/linux-challenges/challenge15/app_data/: 1. `tmp_uploads/` accumulates temporary `.tmp` files that should be automatically deleted once they're older than 7 days (the setup back-dates upload_1..3 by 10 days, so `-mtime +7` matches 3 of the 5 files). 2. A security scan needs to find EVERY file under `app_data/` that is WORLD-WRITABLE (a real, common audit finding - notice `configs/secrets.conf` was accidentally left as such, and so was a second file with a SPACE in its name) and fix their permissions in one safe, batched operation.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Delete `.tmp` files older than 7 days with `-mtime +7 -delete` - and when would you need `xargs` instead?
2. Find all **world-writable** files and fix them with `xargs chmod o-w` using `-print0` / `-0` - why?
3. What is the real danger of a careless `find | xargs rm`, and what safety habit do you follow?

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 15`
Workspace	`~/linux-challenges/challenge15/`

Files created	`~/linux-challenges/challenge15/PROBLEM.md`
	`~/linux-challenges/challenge15/app_data/tmp_uploads/upload_1.tmp ... upload_5.tmp`
	`~/linux-challenges/challenge15/app_data/configs/app.conf`
	`~/linux-challenges/challenge15/app_data/configs/secrets.conf`
	`~/linux-challenges/challenge15/app_data/configs/legacy settings.conf`
	`~/linux-challenges/challenge15/commands.txt`

Deliverable	`~/linux-challenges/challenge15/commands.txt`
Needs	Runs anywhere (any Linux shell).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- Your `.tmp` cleanup command must use `-mtime +7` and `-delete` correctly.
- Your world-writable-fix command must use `-print0` / `xargs -0` correctly, and you must explain exactly why.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

```
bash - ./linux_challenge.sh 15

$ ./linux_challenge.sh 15
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 15 of 20
================================================================================
 CHANGE REQUEST   CHG0030764                          PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Standard change: purge stale tmp uploads and remove
                      world-writable permissions from app_data
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge15/
Read the brief:  cat ~/linux-challenges/challenge15/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge15/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge15/
PROBLEM.md  app_data  commands.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

```
bash - challenge15

$ cd ~/linux-challenges/challenge15
$ ls -l app_data/configs
total 12
-rw-rw-r-- 1 student student 16 Oct 11 13:13 app.conf
-rw-rw-rw- 1 student student 12 Oct 11 13:13 legacy settings.conf
-rw-rw-rw- 1 student student 39 Oct 11 13:13 secrets.conf
# (the setup already back-dated upload_1..3 by 10 days; the output order of find can vary)

# ---- 1) old .tmp files: DRY RUN first, then delete ----
$ find app_data/tmp_uploads -name '*.tmp' -mtime +7 -print
app_data/tmp_uploads/upload_1.tmp
app_data/tmp_uploads/upload_2.tmp
app_data/tmp_uploads/upload_3.tmp
$ find app_data/tmp_uploads -name '*.tmp' -mtime +7 -delete
$ ls -1 app_data/tmp_uploads                        # only the recent files survive
upload_4.tmp
upload_5.tmp

# ---- 2) world-writable files ----
$ find app_data -type f -perm -o+w -print
app_data/configs/legacy settings.conf
app_data/configs/secrets.conf

# WITHOUT -print0 / -0 the space in "legacy settings.conf" breaks xargs:
$ find app_data -type f -perm -o+w | xargs chmod o-w
chmod: cannot access 'app_data/configs/legacy': No such file or directory
chmod: cannot access 'settings.conf': No such file or directory
$ find app_data -type f -perm -o+w -print          # secrets.conf was fixed, the spaced file is STILL open
app_data/configs/legacy settings.conf

# null-delimited names survive any filename:
$ find app_data -type f -perm -o+w -print0 | xargs -0 chmod o-w
$ find app_data -type f -perm -o+w -print | wc -l
0
$ ls -l app_data/configs
total 12
-rw-rw-r-- 1 student student 16 Oct 11 13:13 app.conf
-rw-rw-r-- 1 student student 12 Oct 11 13:13 legacy settings.conf
-rw-rw-r-- 1 student student 39 Oct 11 13:13 secrets.conf
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 15

```
# find, xargs and touch -d are GNU tools on Ubuntu / Debian - every command above is identical.
# One habit worth adding on any distro: -r (--no-run-if-empty) skips the command on empty input.
$ find app_data -type f -perm -o+w -print0 | xargs -0 -r chmod o-w
$ find app_data -type f -perm -o+w -print | wc -l
0
$ find app_data -name '*.nothing' -print0 | xargs -0 echo "RAN-without-r:"
RAN-without-r:                                  # no match, yet it still ran once (risky with rm)
$ find app_data -name '*.nothing' -print0 | xargs -0 -r echo "RAN-with-r:"
                                                # (prints nothing - the command was skipped)

# Age-based cleanup with NO script at all: systemd-tmpfiles (Ubuntu / Debian and RHEL both ship it)
$ sudo tee /etc/tmpfiles.d/app-uploads.conf <<'EOF'
d /srv/app/tmp_uploads 0750 app app 7d
EOF
$ sudo systemd-tmpfiles --clean                 # deletes entries older than 7 days (daily timer too)
# test it with files that are genuinely old; touch -d back-dating may not work because ctime is also checked
```

RESOLUTION INFORMATION			
STATE	Closed - Successful	CLOSE CODE	Successful
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	09-Oct-2026 00:00
CLOSE NOTES	Old `.tmp` files removed with `find -mtime +7 -delete`; world-writable files fixed with `find -print0 | xargs -0 chmod o-w`, including the file with a space in its name.		

KEY TAKEAWAYS

- `-delete` is simpler than `| xargs rm`: no extra process, no quoting problems. Use `xargs` when you must feed files to a **more complex command** or batch them into one invocation.
- `-print0` / `-0` separate names with a NUL byte instead of whitespace, so `legacy settings.conf` stays **one** argument instead of becoming `legacy` and `settings.conf`. The failure above is real: the naive version silently left a world-writable file behind.
- **Danger**: an unset variable or typo (`find "$BASE_DIR/tmp"` -> `find "/tmp"`, or worse `/`) silently widens a destructive command.
- **Habit**: always run the same `find ... -print` first and read the list; use `xargs -p` while testing a new bulk delete on real data.
- **Distro note:** `find`, `xargs` and `touch -d` are the GNU versions on RHEL, Ubuntu / Debian and SUSE, so every command here is identical (Alpine / BusyBox and macOS differ).

CHALLENGE
16

Symbolic & Hard Links - Inodes and What Can Go Wrong

ln -s | ln | inodes | ln -sfn | release pointers & rollback

INCIDENT
INC0042027

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**Deploying v1.1.0 did not switch the live release - "current" pointer misbehaving**		
STATE	In Progress	OPENED	07-Oct-2026 17:50
CALLER	Oliver Grant (Release Management)	CONTACT TYPE	Self-service portal
IMPACT	2 - Medium	URGENCY	2 - Medium
CATEGORY	Release / Deployment	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03 (releases/)	TARGET RESOLUTION	3 business days
ASSIGNMENT GROUP	Cloud & DevOps	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

See ~/linux-challenges/challenge16/DEPLOYMENT_CONTEXT.md - you need to implement a "current release" pointer pattern correctly, and explain exactly why the engineer's earlier hard-link attempt didn't work the way they expected.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Explain **symbolic vs hard link** in terms of inodes - and why a hard link cannot cross filesystems.
2. Why did the hard-link `current` fail as a release pointer?
3. Implement it correctly with a **symlink to the directory** and re-point it with `ln -sfn` - what do `-f` and `-n` do?
4. What is the **rollback** advantage of this pattern?

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 16`
Workspace	`~/linux-challenges/challenge16/`

Files created	`~/linux-challenges/challenge16/PROBLEM.md`
	`~/linux-challenges/challenge16/DEPLOYMENT_CONTEXT.md`
	`~/linux-challenges/challenge16/releases/v1.0.0/app.jar`
	`~/linux-challenges/challenge16/releases/v1.1.0/app.jar`
	`~/linux-challenges/challenge16/commands.txt`

Deliverable	`~/linux-challenges/challenge16/commands.txt`
Needs	Runs anywhere (any Linux shell).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- You must correctly explain the inode-based reasoning for why a hard link doesn't work as a "current version" pointer.
- Your symlink commands must correctly use `ln -sfn` for re-pointing an existing symlink.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

```
bash - ./linux_challenge.sh 16

$ ./linux_challenge.sh 16
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 16 of 20
================================================================================
 INCIDENT   INC0042027                                PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Deploying v1.1.0 did not switch the live release -
                      "current" pointer misbehaving
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge16/
Read the brief:  cat ~/linux-challenges/challenge16/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge16/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge16/
DEPLOYMENT_CONTEXT.md  PROBLEM.md  commands.txt  releases
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

```
bash - challenge16

$ cd ~/linux-challenges/challenge16
# ---- hard link = another directory entry for the SAME inode ----
$ ln releases/v1.0.0/app.jar hard.jar
$ ls -li hard.jar releases/v1.0.0/app.jar
2231118 -rw-r--r-- 2 student student 18 Oct 11 13:13 hard.jar
2231118 -rw-r--r-- 2 student student 18 Oct 11 13:13 releases/v1.0.0/app.jar
# same inode (2231118), link count 2. It is tied to THAT inode - v1.1.0/app.jar is a different inode.
$ rm hard.jar

# ---- correct pattern: symlink the DIRECTORY (a relative target is resolved from the link's folder) ----
$ ln -sfn v1.1.0 releases/current
$ ls -l releases
total 8
lrwxrwxrwx 1 student student    6 Oct 11 13:13 current -> v1.1.0
drwxr-xr-x 2 student student 4096 Oct 11 13:13 v1.0.0
drwxr-xr-x 2 student student 4096 Oct 11 13:13 v1.1.0
$ cat releases/current/app.jar
app version 1.1.0

# ---- instant rollback / "deploy v1.0.0" = re-point the same symlink ----
$ ln -sfn v1.0.0 releases/current
$ cat releases/current/app.jar
app version 1.0.0

# ---- the gotcha -n protects against: plain `ln -s` on an existing directory-symlink ----
$ ln -s v1.1.0 releases/current                  # follows 'current' and creates a link INSIDE v1.0.0/
$ ls -l releases/v1.0.0
total 4
-rw-r--r-- 1 student student 18 Oct 11 13:13 app.jar
lrwxrwxrwx 1 student student  6 Oct 11 13:13 v1.1.0 -> v1.1.0
$ rm releases/v1.0.0/v1.1.0                      # clean up that useless self-pointing link

# ---- the other classic mistake: a relative target written from the wrong folder ----
$ ln -sfn releases/v1.1.0 releases/current
$ cat releases/current/app.jar
cat: releases/current/app.jar: No such file or directory
# 'releases/v1.1.0' is resolved from INSIDE releases/ -> releases/releases/v1.1.0 (broken link)
$ ln -sfn v1.1.0 releases/current                # fixed: relative to the link's own folder
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 16

```
# ln, ls -li and symlink behaviour are identical on every distro. Handy inspection commands:
$ readlink releases/current                      # one hop: what the link literally contains
v1.1.0
$ readlink -f releases/current                   # fully resolved absolute path
/home/student/linux-challenges/challenge16/releases/v1.1.0
$ stat -c '%i %h %n' releases/v1.0.0/app.jar     # inode, hard-link count, name
2231070 1 releases/v1.0.0/app.jar
# Debian / Ubuntu use the SAME symlink idea for "which version is the default" - a chain of links:
$ ls -l /etc/alternatives/editor                 # representative
lrwxrwxrwx 1 root root 9 Oct 11 08:06 /etc/alternatives/editor -> /bin/nano
$ update-alternatives --display editor           # Debian / Ubuntu tool for those links
# RHEL-family names the same tool "alternatives":  alternatives --display editor
```

RESOLUTION INFORMATION

STATE	Resolved	CLOSE CODE	Solved (Permanently)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	07-Oct-2026 21:52
CLOSE NOTES	`current` is now a symlink to the release directory and is re-pointed with `ln -sfn v1.1.0`; rollback shown by re-pointing to v1.0.0.		

KEY TAKEAWAYS

- A **hard link** is another name for the same **inode**, so it cannot cross filesystems (inode numbers mean nothing on another one). A **symlink** stores a **path string** resolved at access time, so it can point anywhere.
- A hard link has **no relationship** to "the newest release": a new version is a different file with a different inode. You would have to re-create the link on every deploy.
- `-f` forces replacement of the existing link; `-n` treats `current` as the link itself rather than following it into the target directory. You need **both**.
- Symlink the **directory** so `releases/current/app.jar` and any future files all switch together as one version.
- **Rollback = one command, milliseconds, no copying.** Copy-into-place deployments must re-copy old files over new ones, slowly and with partial-state risk.
- A relative target is resolved from the **link's folder**, not from where you typed the command - use `v1.1.0` or an absolute path.
- **Distro note:** `ln`, `ls -li` and symlink behaviour are identical on every Linux distro.

CHALLENGE
17

Environment Variables & Shell Profiles - PATH and Login Shells

~/.bashrc | login vs non-login | interactive vs non-interactive | cron PATH

INCIDENT
INC0042080

PRIORITY
P3 - Moderate

SHORT DESCRIPTION	**deploy-tool works over SSH but fails with "command not found" in cron and CI**		
STATE	In Progress	OPENED	08-Oct-2026 08:51
CALLER	Mia Hoffmann (Build & Release)	CONTACT TYPE	Self-service portal
IMPACT	2 - Medium	URGENCY	2 - Medium
CATEGORY	Automation / Environment	BUSINESS SERVICE	CI/CD Pipeline
CONFIGURATION ITEM	build-srv-01	TARGET RESOLUTION	3 business days
ASSIGNMENT GROUP	Cloud & DevOps	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

See ~/linux-challenges/challenge17/deploy_tool_report.md - a real, extremely common source of confusion: something works in an interactive SSH session but not in cron or CI.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Why is `~/.bashrc` never read by cron or CI shells?
2. Compare `~/.bashrc`, `~/.bash_profile` / `~/.profile` and `/etc/environment` for non-interactive contexts.
3. Give **2 concrete fixes** (PATH inside the crontab, and absolute path).
4. Why is hard-coding absolute paths in automation a **good** practice?

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 17`
Workspace	`~/linux-challenges/challenge17/`

Files created	`~/linux-challenges/challenge17/PROBLEM.md`
	`~/linux-challenges/challenge17/deploy_tool_report.md`
	`~/linux-challenges/challenge17/opt/deploy-tool/bin/deploy-tool`
	`~/linux-challenges/challenge17/explanation.md`

Deliverable	`~/linux-challenges/challenge17/explanation.md`
Needs	Runs anywhere (a stand-in tool is created inside the workspace).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- You must correctly explain WHY ~/.bashrc doesn't apply to cron/CI contexts, referencing login vs. non-login / interactive vs. non-interactive shell distinctions.
- You must give at least 2 concrete, correct fixes.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 17

```
$ ./linux_challenge.sh 17
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 17 of 20
================================================================================
 INCIDENT   INC0042080                                PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : deploy-tool works over SSH but fails with "command not
                      found" in cron and CI
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge17/
Read the brief:  cat ~/linux-challenges/challenge17/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge17/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge17/
PROBLEM.md  deploy_tool_report.md  explanation.md  opt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

bash - challenge17

```
$ cd ~/linux-challenges/challenge17
$ TOOL=$PWD/opt/deploy-tool/bin                    # stand-in for /opt/deploy-tool/bin
$ export PATH=$PATH:$TOOL                          # what the engineer put in ~/.bashrc
$ deploy-tool --version                            # interactive shell: works
deploy-tool 2.3.1

# ---- simulate what cron/CI sees: no rc files, minimal environment ----
$ env -i HOME=$HOME bash -c 'deploy-tool --version'
bash: line 1: deploy-tool: command not found
$ echo $?
127

# ---- fix (a): set PATH inside the crontab file itself ----
$ crontab -e
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/opt/deploy-tool/bin
0 3 * * * deploy-tool --version >> /tmp/deploy.log 2>&1
$ env -i HOME=$HOME PATH=/usr/bin:/bin:$TOOL bash -c 'deploy-tool --version'     # same idea, tested here
deploy-tool 2.3.1

# ---- fix (b): absolute path - depends on NOTHING in the environment ----
0 3 * * * /opt/deploy-tool/bin/deploy-tool --version >> /tmp/deploy.log 2>&1
$ env -i $TOOL/deploy-tool --version               # proof, with a completely empty environment
deploy-tool 2.3.1
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 17

```
# Which startup file a LOGIN bash reads depends on the distro (bash uses the first one it finds):
#   RHEL / Rocky / Alma : ~/.bash_profile   (which sources ~/.bashrc)
#   Ubuntu / Debian     : ~/.profile        (sources ~/.bashrc; no ~/.bash_profile by default)
#   SUSE                : first of ~/.bash_profile, ~/.bash_login, ~/.profile
# Ubuntu / Debian's default ~/.bashrc contains a guard near the top that quits for NON-interactive shells:
$ grep -n -A3 'case $- in' ~/.bashrc               # representative output
6:case $- in
7-    *i*) ;;
8-      *) return;;
9-esac
# Anything exported BELOW that guard never runs under cron or CI. The fixes from the main solution -
# PATH= at the top of the crontab, or an absolute path - behave the same on every distro.
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Solved (Permanently)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	08-Oct-2026 11:53
CLOSE NOTES	`PATH` set inside the crontab and the absolute path used for `deploy-tool`; non-interactive shells never read `~/.bashrc`.		

KEY TAKEAWAYS

- `~/.bashrc` is read by **interactive** shells. A cron job or CI step is **non-interactive** (and usually not a login shell), so it never sources `.bashrc` - the PATH edit simply never happens there.
- `~/.bash_profile` / `~/.profile` = **login** shells. `/etc/environment` is read by PAM at login, but cron keeps its own minimal environment and does not honour it consistently.
- Most reliable: do not depend on **any** startup file - set `PATH=` in the crontab/script, or call the binary by **absolute path**.
- In automation, "portable" means **not dependent on an assumed environment**. Same reason you use `/usr/bin/python3` in a systemd `ExecStart=` (Challenge 4).
- **Distro note:** login shells read `~/.bash_profile` on RHEL but `~/.profile` on Ubuntu / Debian, and Ubuntu's default `~/.bashrc` returns early for non-interactive shells - one more reason cron and CI must not depend on it.

CHALLENGE

18

Archiving & Compression - tar, gzip & a Real Backup Strategy

tar -czvf | --exclude | $(date +%F) | tar tzf | partial restore

SERVICE REQUEST (RITM)

RITM0010581

PRIORITY

P3 - Moderate

SHORT DESCRIPTION	**Dated backup of app_to_backup excluding logs, integrity check, and partial restore of config/**		
STATE	Work in Progress	OPENED	08-Oct-2026 10:24
REQUESTED FOR	Ethan Calloway (Application Owner)	REQUESTED BY	Ethan Calloway (Application Owner)
CATALOG ITEM	Backup & Restore Request	REQUEST	REQ0031145
APPROVAL	Approved (Service Owner)	CATALOG TASK	SCTASK0010733
CATEGORY	Backup & Restore	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03 (app_to_backup)	TARGET FULFILMENT	3 business days
ASSIGNMENT GROUP	IT Operations - L2	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

See ~/linux-challenges/challenge18/BACKUP_REQUIREMENTS.md - a realistic, complete backup/restore workflow for ~/linux-challenges/challenge18/app_to_backup/.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Create the dated `.tar.gz`, excluding `logs/` - explain `-c -z -v -f --exclude`.
2. `tar czf` vs `tar cjf` - gzip vs bzip2 trade-off.
3. Verify integrity without extracting (`tar tzf`).
4. Restore only `config/` into a fresh directory - and why you need the exact internal path.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 18`
Workspace	`~/linux-challenges/challenge18/`

Files created	`~/linux-challenges/challenge18/PROBLEM.md`
	`~/linux-challenges/challenge18/app_to_backup/config/app.conf`
	`~/linux-challenges/challenge18/app_to_backup/data/records.db`
	`~/linux-challenges/challenge18/app_to_backup/logs/old.log`
	`~/linux-challenges/challenge18/BACKUP_REQUIREMENTS.md`
	`~/linux-challenges/challenge18/backup_commands.txt`

Deliverable	`~/linux-challenges/challenge18/backup_commands.txt`
Needs	Runs anywhere (any Linux shell).
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- Your archive command must correctly exclude logs/ and embed the current date in the filename.
- Your partial-restore command must correctly target only the config/ subdirectory.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

```
bash - ./linux_challenge.sh 18

$ ./linux_challenge.sh 18
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 18 of 20
================================================================================
 SERVICE REQUEST (RITM)   RITM0010581                 PRIORITY: P3 - Moderate
--------------------------------------------------------------------------------
 Short description  : Dated backup of app_to_backup excluding logs, integrity
                      check, and partial restore of config/
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge18/
Read the brief:  cat ~/linux-challenges/challenge18/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge18/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge18/
BACKUP_REQUIREMENTS.md  PROBLEM.md  app_to_backup  backup_commands.txt
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

```
bash - challenge18

$ cd ~/linux-challenges/challenge18
$ tar -czvf "app-backup-$(date +%F).tar.gz" --exclude='app_to_backup/logs' app_to_backup
app_to_backup/
app_to_backup/config/
app_to_backup/config/app.conf
app_to_backup/data/
app_to_backup/data/records.db
# -c create  -z gzip  -v verbose  -f archive filename  --exclude skip logs/ entirely

$ ls app-backup-*.tar.gz                          # the date in the name comes from $(date +%F)
app-backup-2026-10-11.tar.gz

# ---- verify: list the contents, nothing is written to disk ----
$ tar tzf app-backup-2026-10-11.tar.gz
app_to_backup/
app_to_backup/config/
app_to_backup/config/app.conf
app_to_backup/data/
app_to_backup/data/records.db

# ---- partial restore: ONLY config/ into a fresh location ----
$ mkdir -p restore
$ tar tzf app-backup-2026-10-11.tar.gz | grep config       # discover the EXACT stored path first
app_to_backup/config/
app_to_backup/config/app.conf
$ tar xzf app-backup-2026-10-11.tar.gz -C restore app_to_backup/config
$ find restore -type f
restore/app_to_backup/config/app.conf

# ---- alternative compression (smaller, slower) ----
$ tar -cjf "app-backup-$(date +%F).tar.bz2" --exclude='app_to_backup/logs' app_to_backup
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 18

```
# tar, gzip and date +%F are identical. Minimal Ubuntu / Debian images may lack the other compressors:
$ sudo apt install -y bzip2 xz-utils zstd
$ tar -czvf "app-backup-$(date +%F).tar.gz" --exclude='app_to_backup/logs' app_to_backup     # unchanged
# extra integrity checks on top of tar tzf (nothing is extracted):
$ gzip -t app-backup-2026-10-11.tar.gz && echo "gzip -t: OK"       # the compressed stream is intact
gzip -t: OK
$ sha256sum app-backup-2026-10-11.tar.gz > app-backup-2026-10-11.tar.gz.sha256
$ sha256sum -c app-backup-2026-10-11.tar.gz.sha256                  # re-run later / on another host
app-backup-2026-10-11.tar.gz: OK
# other formats (GNU tar 1.31+, i.e. Ubuntu 22.04+ and Debian 11+):
$ tar -cJf "app-backup-$(date +%F).tar.xz" --exclude='app_to_backup/logs' app_to_backup  # xz
$ tar --zstd -cf "app-backup-$(date +%F).tar.zst" --exclude='app_to_backup/logs' app_to_backup  # zstd
$ tar --zstd -tf app-backup-2026-10-11.tar.zst     # list a zstd archive the same way
```

RESOLUTION INFORMATION

STATE	Resolved	CLOSE CODE	Request fulfilled
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	08-Oct-2026 13:57
CLOSE NOTES	`app-backup-YYYY-MM-DD.tar.gz` created excluding `logs/`, verified with `tar tzf`; `config/` restored to a fresh directory.		

KEY TAKEAWAYS

- `$(date +%F)` expands to `YYYY-MM-DD`, so each day's archive has its own name.
- **gzip** (`czf`) is faster, slightly larger - best for large, frequent (nightly) jobs. **bzip2** (`cjf`) compresses tighter but is slower - better for small, infrequent archives where storage matters more than time.
- `tar tzf` reads through the compressed stream and lists entries without writing files - a fast, safe sanity check that the archive is readable.
- Partial restore matches the **literal stored path** (relative to where `tar` was run), so list first with `tar tzf` if unsure.
- **Distro note:** `tar`, `gzip` and `date +%F` are identical on every distro (`bzip2` may need installing on minimal images).

CHALLENGE
19

Performance Troubleshooting - Load Average, Memory & I/O

uptime | nproc | free -h | vmstat | swap thrashing | I/O wait

INCIDENT
INC0042117

PRIORITY
P2 - High

SHORT DESCRIPTION	**Production server very slow - load 8.42 on 4 cores, swap 3.8/4.0 Gi used, I/O wait ~45%**		
STATE	In Progress	OPENED	08-Oct-2026 11:38
CALLER	Monitoring alert: load average 8.42 on 4 cores	CONTACT TYPE	Monitoring alert
IMPACT	1 - High	URGENCY	2 - Medium
CATEGORY	Performance	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	app-srv-03	TARGET RESOLUTION	8 hours
ASSIGNMENT GROUP	IT Operations - L2	ASSIGNED TO	You (DevOps Engineer)

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

Customers report this production server is "slow." See ~/linux-challenges/challenge19/PERFORMANCE_SNAPSHOT.md for the exact diagnostic commands and output captured live during the incident - work through this as if on a real incident call, explaining your reasoning at each step.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Interpret **load average** vs `nproc` and the trend 6.10 -> 7.95 -> 8.42.
2. Interpret `free -h` - why is heavy **swap** (not "memory mostly used") the red flag?
3. Interpret `wa` **~45%** and tie it to swapping and load in a single causal chain.
4. Give the **immediate** and **long-term** remediation.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 19`
Workspace	`~/linux-challenges/challenge19/`

Files created	`~/linux-challenges/challenge19/PROBLEM.md`
	`~/linux-challenges/challenge19/PERFORMANCE_SNAPSHOT.md`
	`~/linux-challenges/challenge19/ANSWER.md`

Deliverable	`~/linux-challenges/challenge19/ANSWER.md` (written explanation + commands)
Needs	Analysis challenge - the commands run on any Linux shell.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

• You must connect ALL THREE pieces of diagnostic data (load average, swap usage, I/O wait) into ONE coherent causal explanation, not 3 separate, disconnected observations.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

```
bash - ./linux_challenge.sh 19

$ ./linux_challenge.sh 19
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 19 of 20
================================================================================
 INCIDENT   INC0042117                                    PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : Production server very slow - load 8.42 on 4 cores, swap
                      3.8/4.0 Gi used, I/O wait ~45%
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge19/
Read the brief:  cat ~/linux-challenges/challenge19/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge19/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge19/
ANSWER.md  PERFORMANCE_SNAPSHOT.md  PROBLEM.md
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

```
bash - challenge19

$ cd ~/linux-challenges/challenge19
# the figures below come from PERFORMANCE_SNAPSHOT.md (captured on app-srv-03)
$ uptime
 14:32:01 up 45 days,  3:12,  2 users,  load average: 8.42, 7.95, 6.10
$ nproc
4
# load 8.42 on 4 cores = ~2x oversubscribed, and rising (6.10 -> 7.95 -> 8.42 across 15/5/1 min)

$ free -h
              total        used        free      shared  buff/cache   available
Mem:           16Gi        14Gi       200Mi       100Mi       1.7Gi       1.5Gi
Swap:         4.0Gi       3.8Gi       200Mi
# the red flag is SWAP 3.8Gi/4.0Gi, not "14Gi used"

$ vmstat 1 3
procs -----------memory---------- ---swap-- -----io---- -system-- ------cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa st
 6  4  3891200 204800 102400 1740000  120  340   88   210 1400  2200 22 18 15 45  0
 5  5  3891300 198000 102400 1741000  118  355   92   225 1420  2250 21 19 14 46  0
 6  4  3891400 201000 102400 1739500  125  348   90   218 1410  2230 23 17 15 45  0
# si/so > 0 = actively swapping in/out ; wa ~45% = CPU idle, stuck waiting on disk ; b = blocked procs

# ---- immediate: find and relieve the memory hog ----
$ ps aux --sort=-%mem | head -5                    # top line = the biggest memory consumer
$ sudo systemctl restart <offending-service>       # (or kill the process) to relieve pressure now

# ---- long-term: stop it happening again ----
# right-size RAM | fix the memory leak | set memory limits | alert BEFORE swap fills, e.g. Prometheus:
#   (1 - node_memory_SwapFree_bytes / node_memory_SwapTotal_bytes) > 0.5
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 19

```
# uptime, free, vmstat and ps come from procps on every distro. Where the swap lives differs:
$ swapon --show                                  # lists swap on any distro (representative output)
NAME      TYPE SIZE USED PRIO
/swap.img file   4G 3.8G   -2
# Ubuntu's default swap is a swapfile (/swap.img); RHEL's installer usually creates a swap partition or LV.
# iostat needs the sysstat package:  dnf / apt / zypper install sysstat   (see challenge 3)
```

RESOLUTION INFORMATION			
STATE	Resolved	CLOSE CODE	Solved (Work Around)
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	08-Oct-2026 13:13
CLOSE NOTES	Diagnosed swap thrashing as one causal chain; heaviest memory consumer restarted. Right-sizing, leak fix and swap alerting recommended as the permanent fix.		

KEY TAKEAWAYS

- **Load average** counts processes running **or waiting** (CPU *or* I/O) - it is not "% CPU". 8.42 on 4 cores is overloaded, and the **rising trend** says it is getting worse, not recovering.
- "Most memory used" is normal (Linux fills spare RAM with page cache). **Heavy swap** is not: disk is orders of magnitude slower than RAM, so every swapped-out page access stalls a process.
- **One causal chain:** RAM exhausted -> heavy swapping -> processes wait on disk -> high `wa` -> processes pile up -> high load. The load is **I/O wait from swap thrashing**, not genuine CPU-bound work.
- Killing one process fixes the **symptom**. Without right-sizing, a leak fix or alerting, the same incident returns the next time memory pressure builds.
- **Distro note:** the diagnosis is distro-independent; only where swap lives differs (`swapon --show` lists it), and `iostat` needs the `sysstat` package.

CHALLENGE 20

Capstone - Provision & Harden a Production Linux Server

end-to-end runbook | ordering | least privilege | systemd | firewalld | logrotate | SSH

CHANGE REQUEST
CHG0030778

PRIORITY
P2 - High

SHORT DESCRIPTION	**New production server build for billing-service: provision and harden end to end**		
STATE	Implement	OPENED	08-Oct-2026 13:02
TYPE	Normal	RISK	High
REQUESTED BY	Elena Petrova (IT Operations)	APPROVAL	Approved (CAB)
CATEGORY	Server Build	BUSINESS SERVICE	Billing Platform
CONFIGURATION ITEM	billing-srv-new (RHEL)	CHANGE TASK	CTASK0031053
PLANNED START	09-Oct-2026 22:00	PLANNED END	10-Oct-2026 00:00
ASSIGNMENT GROUP	Cloud & DevOps	ASSIGNED TO	You (DevOps Engineer)
JUSTIFICATION	A bare RHEL/CentOS VM must become the production home of billing-service. Steps are ordered; swapping two of them breaks the build.		

DESCRIPTION - WHAT WE ARE TRYING TO SOLVE

This is the real one - a genuine "here's a bare VM, make it production- ready" exercise, combining every skill from Challenges 1-19 into one coherent runbook. Plan before you type: talk through your PLAN before writing any commands, then write the actual runbook.

Read ~/linux-challenges/challenge20/REQUIREMENTS.md - assume a fresh, bare Linux VM.

THE QUESTION - WORK REQUIRED ON THIS TICKET

1. Create a **non-root** `billing` account with no login shell.
2. Set **ownership / permissions** (nothing world-writable).
3. Install **pinned** dependencies; write + enable the **systemd unit**.
4. Firewall: **only** the app port, **only** from the internal CIDR.
5. **logrotate**, nightly **backup timer** (tar, excluding logs), **SSH hardening**, **current-release symlink**, **disk/inode monitor**.

Summary of the work required. The full ticket wording, with hints, is in `PROBLEM.md` in the workspace.

FILES, LOCATION & HOW TO RUN

Run	`./linux_challenge.sh 20`
Workspace	`~/linux-challenges/challenge20/`

Files created	`~/linux-challenges/challenge20/PROBLEM.md`
	`~/linux-challenges/challenge20/REQUIREMENTS.md`
	`~/linux-challenges/challenge20/RUNBOOK.md`

Deliverable	`~/linux-challenges/challenge20/RUNBOOK.md`
Needs	root on a fresh server - RHEL-family steps, plus an Ubuntu / Debian variant.
Distros	RHEL / Rocky / Alma: main solution. **Ubuntu / Debian: the orange block right after it.**

CLOSURE CRITERIA - THIS TICKET CAN BE CLOSED WHEN

- The runbook must be genuinely ORDERED in a way that makes sense (e.g. create the user BEFORE setting file ownership to that user; install the app BEFORE writing a systemd unit that starts it).
- Every step must tie back to a specific, named security or reliability principle from the earlier challenges - not just "run this command because it's common practice."
- Be ready to explain, live, why each step is ordered where it is, and what would break if two steps were swapped.

WHAT THE SCRIPT PRINTS - REAL OUTPUT (MIDDLE OF THE TICKET SHORTENED)

bash - ./linux_challenge.sh 20

```
$ ./linux_challenge.sh 20
================================================================================
 LINUX CHALLENGES SIMULATOR                                  CHALLENGE 20 of 20
================================================================================
 CHANGE REQUEST   CHG0030778                               PRIORITY: P2 - High
--------------------------------------------------------------------------------
 Short description  : New production server build for billing-service:
                      provision and harden end to end
[ ... state, caller, description, work required, closure criteria ... ]
================================================================================

Workspace:       ~/linux-challenges/challenge20/
Read the brief:  cat ~/linux-challenges/challenge20/PROBLEM.md
Files created:   ls ~/linux-challenges/challenge20/
Deliverable(s):  fill in the template file(s) named in the ticket above

$ ls ~/linux-challenges/challenge20/
PROBLEM.md  REQUIREMENTS.md  RUNBOOK.md
```

SOLUTION - AS YOU WOULD TYPE IT ON THE TERMINAL

Runbook 1/3 - identity, files, packages, release

```
# STEP 1 - service account FIRST (so every later chown / User= has a valid target)  [Challenge 2]
$ sudo useradd -r -s /sbin/nologin -d /opt/billing-service billing

# STEP 2 - directory tree + ownership; nothing world-writable  [Challenge 1]
$ sudo mkdir -p /opt/billing-service/{releases/v1.0.0,data} /var/log/billing-service /backups
$ sudo chown -R billing:billing /opt/billing-service /var/log/billing-service
$ sudo chmod -R 750 /opt/billing-service

# STEP 3 - dependencies, pinned  [Challenge 9]
$ sudo yum install -y python3 libfoo-1.4.2 yum-plugin-versionlock
$ sudo yum versionlock add libfoo-1.4.2

# STEP 4 - first release + "current" symlink (before the unit, so ExecStart can use it)  [Challenge 16]
$ sudo cp worker.py /opt/billing-service/releases/v1.0.0/
$ sudo ln -sfn /opt/billing-service/releases/v1.0.0 /opt/billing-service/current
$ sudo chown -R billing:billing /opt/billing-service/releases
$ sudo chown -h billing:billing /opt/billing-service/current
```

SOLUTION (CONTINUED)

Runbook 2/3 - service, firewall, logs

```
# STEP 5 - systemd unit, runs as billing, restarts on failure  [Challenge 4]
$ sudo tee /etc/systemd/system/billing-service.service <<'EOF'
[Unit]
Description=Billing Service
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/billing-service/current/worker.py
ExecReload=/bin/kill -HUP $MAINPID
StandardOutput=append:/var/log/billing-service/app.log
StandardError=inherit
User=billing
Group=billing
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
# ExecReload makes "systemctl reload" work; StandardOutput=append: writes to the file logrotate watches
# The step 7 postrotate reload only does something if worker.py handles SIGHUP by reopening its log file
# If it does not, use copytruncate instead of postrotate in the step 7 logrotate block
```

Runbook 2/3 - service, firewall, logs

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now billing-service
$ ss -tlnp | grep 8080                          # confirm what the app REALLY listens on

# STEP 6 - firewall AFTER the port is known: one port, one CIDR  [Challenge 8]
$ sudo firewall-cmd --permanent --zone=public --add-rich-rule=\
> 'rule family="ipv4" source address="10.0.0.0/16" port port="8080" protocol="tcp" accept'
$ sudo firewall-cmd --reload

# STEP 7 - log rotation  [Challenge 13]
$ sudo tee /etc/logrotate.d/billing-service <<'EOF'
/var/log/billing-service/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    postrotate
        systemctl reload billing-service > /dev/null 2>&1 || true
    endscript
}
EOF
```

SOLUTION (CONTINUED)

Runbook 3/3 - backup, SSH, monitoring

```
# STEP 8 - nightly backup via systemd TIMER (observable; logs excluded)  [Challenges 12 + 18]
$ sudo tee /opt/billing-service/backup.sh <<'EOF'
#!/bin/bash
set -euo pipefail
tar czf /backups/billing-$(date +%F).tar.gz -C /opt/billing-service --exclude='*.log' data
EOF
# -C + a relative path also removes the "Removing leading /" warning
$ sudo chmod 750 /opt/billing-service/backup.sh
$ sudo tee /etc/systemd/system/billing-backup.service <<'EOF'
[Unit]
Description=Nightly billing-service backup

[Service]
Type=oneshot
ExecStart=/opt/billing-service/backup.sh
EOF
$ sudo tee /etc/systemd/system/billing-backup.timer <<'EOF'
[Unit]
Description=Nightly backup at 02:00

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target
EOF
$ sudo systemctl daemon-reload && sudo systemctl enable --now billing-backup.timer

# STEP 9 - SSH hardening (verify key login in a SECOND session FIRST)  [Challenge 14]
$ sudo tee /etc/ssh/sshd_config.d/00-hardening.conf <<'EOF'   # first value wins; 00- beats cloud-init
PermitRootLogin no
PasswordAuthentication no
X11Forwarding no
MaxAuthTries 3
ClientAliveInterval 300
ClientAliveCountMax 2
EOF
$ sudo sshd -t && sudo systemctl reload sshd          # Ubuntu / Debian: ssh
$ sudo sshd -T | grep -i -E 'permitrootlogin|passwordauthentication'   # confirm the effective values
```

Runbook 3/3 - backup, SSH, monitoring

```
# STEP 10 - disk + inode monitor, hourly; cron mails MAILTO only on failure  [Challenges 5 + 10]
$ sudo tee /opt/billing-service/disk_check.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail

THRESHOLD="${1:-90}"
MODE="${2:-space}"                    # "space" (default) or "inodes"
OVER_THRESHOLD=0

check_line() {
  local usage="$1" mount="$2"
  if [ "$usage" -gt "$THRESHOLD" ]; then
    echo "WARNING: $mount ($MODE) is at ${usage}% - over threshold!"
    return 1
  fi
  return 0
}

DF_FLAGS="-P"
[ "$MODE" = "inodes" ] && DF_FLAGS="-Pi"

while read -r _fs _total _used _free pct mount; do
  usage="${pct%\%}"
  [[ "$usage" =~ ^[0-9]+$ ]] || continue
  if ! check_line "$usage" "$mount"; then
    OVER_THRESHOLD=1
  fi
done < <(df $DF_FLAGS -x tmpfs -x devtmpfs -x squashfs | tail -n +2)

echo "Disk check ($MODE) complete."
exit "$OVER_THRESHOLD"
EOF
$ sudo chown root:billing /opt/billing-service/disk_check.sh
$ sudo chmod 750 /opt/billing-service/disk_check.sh
$ sudo tee /etc/cron.d/billing-disk-check <<'EOF'
MAILTO=oncall@company.com
0 * * * * root /opt/billing-service/disk_check.sh 90 >/dev/null || logger -p user.crit "billing: disk SPACE threshold breached"
5 * * * * root /opt/billing-service/disk_check.sh 90 inodes >/dev/null || logger -p user.crit "billing: INODE threshold breached"
EOF
# MAILTO only works if a mail agent (postfix or mailx) is installed; the logger line is the fallback
$ systemctl list-timers billing-backup.timer       # final check: next backup is scheduled
```

UBUNTU / DEBIAN

Same task, Ubuntu-style commands - run this block on an Ubuntu or Debian VM

also Linux Mint Pop!_OS, Raspberry Pi OS

Ubuntu / Debian equivalent - challenge 20 (only the steps that differ)

```
# Steps 2, 4, 5 (unit file), 7 (logrotate) and 8 (backup timer) are IDENTICAL on Ubuntu / Debian.
# STEP 1 - service account: the no-login shell path differs
$ sudo useradd -r -s /usr/sbin/nologin -d /opt/billing-service billing

# STEP 3 - dependencies, pinned: apt instead of yum   (libfoo is the fictional pinned package)
$ sudo apt update
$ sudo apt install -y python3 libfoo=1.4.2-1
$ sudo apt-mark hold libfoo

# STEP 6 - firewall: ufw instead of firewalld. SSH first, then one port from one CIDR
$ sudo ufw allow 22/tcp
$ sudo ufw allow from 10.0.0.0/16 to any port 8080 proto tcp
$ sudo ufw enable

# STEP 9 - SSH hardening: same drop-in + sshd -t, but the service is called ssh
$ sudo sshd -t && sudo systemctl reload ssh
$ sudo sshd -T | grep -i -E 'permitrootlogin|passwordauthentication'     # no drop-in overriding you?

# STEP 10 - disk monitor: the cron service is "cron" (RHEL: crond) - make sure it runs
$ sudo systemctl enable --now cron
```

RESOLUTION INFORMATION			
STATE	Closed - Successful	CLOSE CODE	Successful
RESOLVED BY	You (DevOps Engineer)	RESOLVED ON	10-Oct-2026 00:00
CLOSE NOTES	Server built in 10 ordered steps: service account, permissions, pinned packages, release symlink, systemd unit, firewall, logrotate, backup timer, SSH hardening, disk monitor.		

KEY TAKEAWAYS

- **Why this order:** the user exists *before* `chown` / `User=` reference it (otherwise chown fails); the app + symlink exist *before* the unit starts them; the firewall opens *after* you know the real port.
- Step 4 (symlink) sits ahead of the unit so `ExecStart` can use `current/` - future deploys are then an atomic `ln -sfn` plus a restart.
- **SSH hardening and disk monitoring** are independent of the app, so they go last as a deliberate "harden and monitor" pass - after the service works, and never before key login is proven.
- Each step maps to a principle: **least privilege** (steps 1, 2, 6), **reproducibility** (3, 4), **resilience** (5, 7, 8), **observability** (8, 10), **attack-surface reduction** (9).
- **Distro note:** the order and the reasons do not change on Ubuntu / Debian - only the tools: `yum` -> `apt` + `apt-mark hold`, `firewalld` -> `ufw` (SSH rule first), `sshd` -> `ssh`, `crond` -> `cron`, `/sbin/nologin` -> `/usr/sbin/nologin`.

Command Cheat Sheet

Every command used in the challenges, grouped by challenge for quick reference. Anything in `<angle brackets>` is a placeholder to replace. Run privileged commands with `sudo`.

01	File Permissions & Ownership - chmod, chown & Special Bits	INC0041831
`ls -ld <path>`	Show mode / owner / group of the directory or file itself
`groupadd <group>`	Create a group
`chown :finance <dir>`	Change the group only
`chown root:backup-operators <file>`	Set owner and group in one go
`chmod 3775 <dir>`	setgid (2) + sticky (1) + rwxrwxr-x
`chmod 3770 <dir>`	setgid + sticky, no access for "others" (shows s / T)
`chmod 750 <file>`	owner rwx, group r-x, others none
`find / -xdev -perm -4000 -type f`	Audit: list every setuid file
`getent group <group>`	Confirm a group exists (any distro)
`stat -c "%A %a %U:%G %n" <path>`	Mode, octal and owner in one line

02	User & Group Management - useradd, sudoers & Least Privilege	RITM0010505
`useradd -m -s /bin/bash -G developers -c "Name" user`	Create user with home, shell, supplementary group
`id <user>`	Confirm uid / gid / groups
`passwd <user>`	Set the temporary password
`chage -d 0 <user>`	Expire password: forced change at next login
`chage -l <user>`	Show password ageing info
`visudo -f /etc/sudoers.d/<file>`	Edit a sudoers drop-in safely
`visudo -cf /etc/sudoers.d/<file>`	Validate sudoers syntax
`sudo -l -U <user>`	List exactly what the user may run
`usermod -aG developers <user>`	Add a user to a group later (-a = append) - all distros
`adduser <user>`	Debian / Ubuntu interactive wrapper around useradd

03	Process Management - ps, kill, Signals, nice & Job Control	INC0041888
`ps -o ppid= -p <pid>`	Find the parent of a (zombie) process
`ps -eo user,pid,ppid,%cpu,stat,args`	Process list with parent PID and state
`kill -15 <pid>`	SIGTERM - polite stop, app can clean up
`kill -9 <pid>`	SIGKILL - forced stop, last resort
`renice -n 10 -p <pid>`	Lower priority (niceness -20 ... +19)
`iostat -x 1 3`	Disk utilisation - why a process sits in D state
`dmesg -T | tail`	Kernel messages: I/O errors, NFS timeouts
`Ctrl+Z | bg %1 | jobs -l`	Suspend, resume in background, list jobs
`disown -h %1`	Stop the shell sending SIGHUP to the job
`nohup <cmd> > job.log 2>&1 &`	Start a job that survives logout
`apt install sysstat | dnf install sysstat`	Get iostat on Ubuntu / RHEL (SUSE: zypper install sysstat)

04 systemd Service Management - Writing a Custom Unit File — CHG0030636

`useradd -r -s /sbin/nologin <svc>`	Create a non-login service account
`systemd-analyze verify <unit>`	Syntax-check a unit file
`cp <unit> /etc/systemd/system/`	Install the unit file
`systemctl daemon-reload`	Re-read unit files after any change
`systemctl enable --now <unit>`	Start on boot AND start right now
`systemctl status <unit> --no-pager`	Show state, PID and recent log lines
`systemctl is-active <unit>`	Script-friendly active / inactive
`systemctl show -p MainPID --value <unit>`	Print the main PID only
`Restart=on-failure RestartSec=5`	Unit options: restart after a crash, 5 s pause
`useradd -r -s /usr/sbin/nologin <svc>`	Ubuntu / Debian path of the no-login shell

05 Disk & Filesystem Management - df, du, mount & /etc/fstab — INC0041904

`df -h <path>`	Disk space used / free
`df -i <path>`	Inode usage - catches "full" disks with free space
`find <dir> -xdev | wc -l`	Count entries on one filesystem (inode hunt)
`blkid /dev/sdb1`	Get the UUID of a partition
`mkdir -p /opt/app-logs`	Create the mount point
`cp /etc/fstab /etc/fstab.bak`	Back up before editing
`tee -a /etc/fstab (UUID=<id> <mnt> ext4 defaults 0 2)`	Add a permanent mount by UUID
`mount -a`	Mount everything in fstab - also validates the new line
`findmnt <mountpoint>`	Confirm the mount
`lsblk -f`	UUID + filesystem of every disk in one view
`findmnt --verify`	Validate fstab without mounting
`df -h -x tmpfs -x squashfs`	Hide snap / tmpfs mounts (Ubuntu)

06 LVM - Extending a Logical Volume Without Downtime — CHG0030686

`vgs <vg>`	Show VG size and free space
`df -h /data`	Check filesystem size before / after
`lvextend -l +100%FREE /dev/<vg>/<lv>`	Grow the LV into all free space
`lvextend -r -l +100%FREE /dev/<vg>/<lv>`	Shortcut: grow LV and filesystem together
`resize2fs /dev/<vg>/<lv>`	Grow ext4 online to match the LV (XFS: xfs_growfs)
`pvcreate /dev/sdc`	Initialise a new disk for LVM
`vgextend <vg> /dev/sdc`	Add the new PV to the volume group
`findmnt -no FSTYPE /data`	Which filesystem? ext4 -> resize2fs, xfs -> xfs_growfs
`xfs_growfs <mountpoint>`	Grow XFS online (RHEL default) - takes the mount point

07	Networking Fundamentals - ip, ss, DNS & Connectivity Troubleshooting	INC0041939
`ss -tlnp | grep 8080`	Listening TCP sockets; read the Local Address column
`systemctl restart <service>`	Apply the changed bind address
`nc -zv <host> <port>`	Test a port from another host (refused vs timed out)
`traceroute -n <host>`	Find where the path stops
`firewall-cmd --list-all`	Check the host firewall layer
`127.0.0.1 vs 0.0.0.0`	Loopback only (local) vs all interfaces (reachable)
`ufw status verbose`	Ubuntu / Debian host-firewall status
`nft list ruleset`	What the kernel really enforces (any distro)

08	Firewall Management - firewalld / iptables Least-Privilege Rules	RITM0010549
`firewall-cmd --permanent --zone=public --add-rich-rule='rule family="ipv4" source address="<cidr>" port port="<p>" protocol="tcp" accept'`	Open one port to one source range only
`firewall-cmd --reload`	Load the permanent config into the running firewall
`firewall-cmd --list-rich-rules`	Verify the rich rules
`firewall-cmd --list-ports`	Confirm nothing is open globally
`10.0.0.0/16 | 10.0.5.20/32`	/16 = whole internal block, /32 = exactly one host
`ufw allow from 10.0.0.0/16 to any port 443 proto tcp`	Ubuntu / Debian: one port from one network
`ufw allow from 10.0.5.20 to any port 8080 proto tcp`	Ubuntu / Debian: one port from one host (bare IP = /32)
`ufw enable | ufw status numbered`	ufw ships inactive; allow SSH first, then enable and verify
`iptables -A INPUT -p tcp -s <cidr> --dport <p> -j ACCEPT`	Raw iptables: ACCEPT rules must precede the DROP

09	Package Management - Repos, Dependency Conflicts & Pinning	INC0041960
`yum list --showduplicates <pkg>`	Installed and available versions
`yum downgrade -y <pkg>-<version>`	Go back to a known-good version
`yum downgrade ./<pkg>.rpm`	Downgrade from a local RPM when the repo dropped it
`yum install -y yum-plugin-versionlock`	Install the pinning plugin
`yum versionlock add <pkg>-<version>`	Pin one package, everything else updates
`yum versionlock list`	Show pinned packages
`apt-mark hold <pkg>`	Debian / Ubuntu equivalent of a pin
`docker run --rm <image> --version`	Isolate a tool that needs a different library version
`apt-cache policy <pkg> | apt list -a <pkg>`	Ubuntu / Debian: installed, candidate and available versions
`apt install <pkg>=<ver> --allow-downgrades`	Ubuntu / Debian: downgrade to an exact version
`apt-mark hold <pkg> | showhold | unhold <pkg>`	Ubuntu / Debian: pin, list, unpin
`dnf versionlock add <pkg>-<ver>`	RHEL 8/9: pin (python3-dnf-plugin-versionlock)
`zypper addlock <pkg> | locks | removelock <pkg>`	SUSE: pin, list, unpin
`podman run --rm <image>`	RHEL 8/9 equivalent of docker run

10	Shell Scripting Fundamentals - A Real Production-Style Script	PRB0040095
`set -euo pipefail`	Stop on error, unset variable, or failed pipe stage
`THRESHOLD="${1:-90}"`	Optional argument with default 90
`df -P | tail -n +2`	Portable one-line-per-filesystem output, header removed
`while read -r _fs _sz _u _a pct mount; do ... done < <(df -P)`	Parse columns; last variable keeps spaces; loop stays in same shell
`usage="${pct%\%}"`	Strip the trailing % sign
`check_line() { ... return 1; }`	Function that reports through its return code
`exit "$OVER_THRESHOLD"`	Exit 1 on breach, 0 otherwise
`chmod +x disk_check.sh`	Make the script executable
`PATH=$PWD/fake:$PATH ./disk_check.sh 85`	Test with a fake df on the PATH
`readlink -f /bin/sh`	dash (Ubuntu / Debian) vs bash (RHEL)
`df -P -x tmpfs -x squashfs -x devtmpfs`	Skip snap / tmpfs mounts in the disk check

11	Text Processing - grep, awk, sed & the Unix Pipeline	INC0041978
`grep -c '" 500 ' access.log`	Count lines with status 500
`grep <path> f | grep '" 500 ' | awk '{print $1}' | sort | uniq -c | sort -rn`	Ranked list of source IPs for a failing endpoint
`awk -F'"' '{print $2}' access.log`	Extract the quoted request (METHOD PATH PROTO)
`awk '{print $1, $2}' | sort | uniq -c | sort -rn`	Count and rank unique METHOD + PATH pairs
`sed -E 's/^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+/REDACTED/' f`	Redact the leading IP address
`head -n 3 <file>`	Peek at the first lines
`zcat -f access.log*`	Read the current and rotated .gz logs together
`zgrep -c "<pattern>" *.gz`	grep inside .gz files

12	Cron & Scheduled Tasks - crontab and systemd Timers	RITM0010568
`crontab -e | crontab -l`	Edit / list your cron jobs
`30 2 * * 1-5 <script>`	cron: 02:30, Monday to Friday
`echo "<script>" | at 23:45`	Run once at 23:45
`atq`	List pending at jobs
`OnCalendar=Mon..Fri 02:30:00`	Timer schedule expression
`Persistent=true`	Run a missed timer as soon as the host is back
`systemd-analyze calendar "<expr>"`	Validate a calendar expression
`systemctl enable --now <name>.timer`	Enable and start the timer
`systemctl list-timers <name>.timer`	Last run / next run
`journalctl -u <name>.service`	Run history incl. failures
`systemctl status cron`	Ubuntu / Debian / SUSE cron service (RHEL: crond)
`apt install at && systemctl enable --now atd`	Install and start atd (dnf / zypper on other distros)

13 Log Management - journalctl, /var/log & logrotate — INC0041994

`logrotate -d <conf>`	Dry run: show what would happen
`logrotate -f <conf>`	Force a rotation now
`daily / rotate 14 / compress / delaycompress`	logrotate: daily, keep 14, gzip, compress one cycle later
`postrotate ... endscript`	Reload the app so it reopens the log file
`copytruncate`	Fallback when the app cannot reopen logs (may lose lines)
`journalctl -u <unit>`	All logs for a service
`journalctl -u <unit> --since "22:00" --until "22:30"`	Incident time window
`journalctl -u <unit> -f`	Follow live
`journalctl -u <unit> -p err`	Errors and worse only
`tail -f /var/log/syslog | /var/log/auth.log`	Ubuntu / Debian system and auth logs
`tail -f /var/log/messages | /var/log/secure`	RHEL-family system and auth logs

14 SSH Configuration & Hardening - Key-Based Auth Done Right — CHG0030734

`ssh-keygen -t ed25519 -C "<comment>"`	Create a modern key pair
`ssh-copy-id -i ~/.ssh/id_ed25519.pub user@host`	Install the public key on the server
`chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys`	Permissions sshd insists on
`ssh -i <key> -o PasswordAuthentication=no user@host`	Prove key login works (second terminal)
`PermitRootLogin no`	sshd_config: no direct root login
`PasswordAuthentication no`	sshd_config: keys only
`X11Forwarding no | MaxAuthTries 3`	sshd_config: less attack surface, fewer guesses
`ClientAliveInterval 300 | ClientAliveCountMax 2`	sshd_config: drop dead sessions
`sshd -t`	Validate config before reload
`systemctl reload sshd`	Apply without dropping sessions
`systemctl reload ssh`	Ubuntu / Debian service name (RHEL / SUSE: sshd)
`daemon-reload && restart ssh.socket`	Ubuntu 22.10+: needed after changing the SSH port
`sshd -T | grep -i <option>`	Effective sshd value after drop-ins in sshd_config.d

15 find & xargs - Safe, Efficient Batch File Operations — CHG0030764

`find <dir> -name '*.tmp' -mtime +7 -print`	Dry run: list files older than 7 days
`find <dir> -name '*.tmp' -mtime +7 -delete`	Delete them
`find <dir> -type f -perm -o+w -print`	List world-writable files
`find <dir> -type f -perm -o+w -print0 | xargs -0 chmod o-w`	Fix them safely, spaces in names included
`xargs -p`	Prompt before each run while testing
`touch -d '10 days ago' <file>`	Back-date a file to test age filters
`xargs -0 -r <cmd>`	Skip the command when find matched nothing
`systemd-tmpfiles --clean`	Age-based cleanup from /etc/tmpfiles.d

16 Symbolic & Hard Links - Inodes and What Can Go Wrong — INC0042027

`ln <file> <hardlink>`	Hard link: second name for the same inode
`ls -li <files>`	Show inode numbers and link count
`ln -sfn v1.1.0 releases/current`	Create or re-point a directory symlink (-f replace, -n do not follow)
`ls -l releases`	See where current points
`cat releases/current/app.jar`	Confirm which version is live
`ln -sfn v1.0.0 releases/current`	Rollback in one command
`readlink -f <link>`	Fully resolved target path
`update-alternatives --display <name>`	Debian symlink chain (RHEL: alternatives)

17 Environment Variables & Shell Profiles - PATH and Login Shells — INC0042080

`export PATH=$PATH:<dir>`	Add to PATH (put it in ~/.bashrc: interactive shells only)
`env -i HOME=$HOME bash -c '<cmd>'`	Simulate the bare environment cron / CI gets
`PATH=/usr/bin:/bin:<dir>`	First line of a crontab: set PATH for all jobs
`/opt/deploy-tool/bin/deploy-tool`	Absolute path - needs no environment at all
`env -i <abs-path-to-tool> --version`	Prove it works with a completely empty environment
`grep -n -A3 'case $- in' ~/.bashrc`	Ubuntu / Debian: find the non-interactive guard in ~/.bashrc

18 Archiving & Compression - tar, gzip & a Real Backup Strategy — RITM0010581

`tar -czvf "app-backup-$(date +%F).tar.gz" --exclude=<dir> <src>`	Dated gzip archive, excluding a folder
`tar tzf <archive>`	List contents without extracting (integrity check)
`tar tzf <archive> | grep config`	Find the exact stored path
`mkdir -p restore`	Fresh restore target
`tar xzf <archive> -C restore <path>`	Restore one folder only
`tar -cjf <name>.tar.bz2 ...`	bzip2 variant: smaller, slower
`gzip -t <file.gz>`	Test a gzip archive without extracting
`sha256sum -c <file>.sha256`	Verify a checksum
`tar --zstd -cf x.tar.zst ...`	zstd compression (GNU tar 1.31+)

19 Performance Troubleshooting - Load Average, Memory & I/O — INC0042117

`uptime`	Load average over 1 / 5 / 15 minutes
`nproc`	CPU core count to compare against load
`free -h`	Memory and swap usage
`vmstat 1 3`	si/so (swapping), wa (I/O wait), b (blocked)
`ps aux --sort=-%mem | head -5`	Top memory consumers
`systemctl restart <service>`	Relieve pressure from the offender
`swapon --show`	List swap devices / files on any distro

20	Capstone - Provision & Harden a Production Linux Server	CHG0030778
`useradd -r -s /sbin/nologin -d <dir> <user>`	Service account first
`mkdir -p <a>/{releases/v1.0.0,data}`	Build the directory tree in one command
`chown -R billing:billing <dir> | chmod -R 750 <dir>`	Ownership and permissions
`yum install -y <pkg>-<ver> yum-plugin-versionlock`	Install pinned dependencies
`ln -sfn <release> <dir>/current`	Release symlink
`systemctl daemon-reload && systemctl enable --now <unit>`	Install and start the unit
`firewall-cmd --permanent --add-rich-rule=... && firewall-cmd --reload`	One port, one CIDR
`logrotate config + postrotate reload`	Daily, 14 kept, compressed
`<name>.timer + <name>.service (OnCalendar, Persistent)`	Observable nightly backup
`tee /etc/ssh/sshd_config.d/00-hardening.conf sshd -t && systemctl reload sshd; sshd -T`	Harden SSH via a drop-in (first value wins)
`chown root:billing <f> && chmod 750 <f>`	Owner and mode for disk_check.sh
`/etc/cron.d/<name> (MAILTO=...)`	Hourly space + inode check, logger fallback

Distro Command Map

The same task on the three families you will meet most. RHEL-family = RHEL, Rocky, Alma, CentOS Stream, Fedora. Anything in `<angle brackets>` is a placeholder. Commands were checked against current distro documentation; they were not executed in the build environment.

TASK	RHEL / Rocky / Alma / Fedora	UBUNTU / DEBIAN	SUSE / openSUSE
PACKAGES			
Install a package	`sudo dnf install <pkg>`	`sudo apt install <pkg>`	`sudo zypper install <pkg>`
Refresh + upgrade	`sudo dnf upgrade`	`sudo apt update && sudo apt upgrade`	`sudo zypper refresh && sudo zypper update`
List versions	`dnf list --showduplicates <pkg>`	`apt-cache policy <pkg> (or apt list -a <pkg>)`	`zypper search -s <pkg>`
Downgrade	`sudo dnf downgrade <pkg>-<ver>`	`sudo apt install <pkg>=<ver> --allow-downgrades`	`sudo zypper install --oldpackage <pkg>=<ver>`
Pin a package	`sudo dnf versionlock add <pkg>`	`sudo apt-mark hold <pkg>`	`sudo zypper addlock <pkg>`
List / remove pins	`dnf versionlock list | delete <pkg>`	`apt-mark showhold | unhold <pkg>`	`zypper locks | removelock <pkg>`
Install a local file	`sudo dnf install ./x.rpm`	`sudo apt install ./x.deb (or dpkg -i)`	`sudo zypper install ./x.rpm`
Which package owns a file	`rpm -qf <file>`	`dpkg -S <file>`	`rpm -qf <file>`
Run a container	`podman run --rm <image>`	`docker run --rm <image> (apt install docker.io)`	`podman run --rm <image> (or docker)`
FIREWALL AND NETWORK			
Default firewall tool	`firewalld (firewall-cmd)`	`ufw (inactive until enabled)`	`firewalld (firewall-cmd)`
Allow a port from one CIDR	`firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="10.0.0.0/16" port port="443" protocol="tcp" accept'`	`ufw allow from 10.0.0.0/16 to any port 443 proto tcp`	`same as RHEL (firewalld)`
Apply and verify	`firewall-cmd --reload | --list-rich-rules`	`ufw status numbered (rules apply at once and persist)`	`same as RHEL (firewalld)`
Install nc (netcat)	`sudo dnf install nmap-ncat`	`sudo apt install netcat-openbsd`	`sudo zypper install netcat-openbsd`
SERVICES AND LOGS			
SSH service	`sshd`	`ssh (Ubuntu 22.10+: also ssh.socket)`	`sshd`
Cron service	`crond`	`cron`	`cron`
Install at + start atd	`sudo dnf install at && sudo systemctl enable --now atd`	`sudo apt install at && sudo systemctl enable --now atd`	`sudo zypper install at && sudo systemctl enable --now atd`
System + auth logs	`/var/log/messages | /var/log/secure`	`/var/log/syslog | /var/log/auth.log (if rsyslog is installed; Debian 12+ often has only the journal, so use journalctl)`	`journalctl (or /var/log/messages with rsyslog)`
No-login shell path	`/sbin/nologin`	`/usr/sbin/nologin`	`/sbin/nologin`
STORAGE			
Install LVM tools	`sudo dnf install lvm2`	`sudo apt install lvm2`	`sudo zypper install lvm2`
Grow a filesystem	`xfs_growfs <mnt> (XFS default) | resize2fs <dev> (ext4)`	`resize2fs <dev> (ext4 default) | xfs_growfs <mnt> (XFS)`	`xfs_growfs <mnt> | resize2fs <dev> | btrfs filesystem resize`
Install iostat	`sudo dnf install sysstat`	`sudo apt install sysstat`	`sudo zypper install sysstat`
SHELL, USERS AND SECURITY			
/bin/sh is	`bash`	`dash (run bash scripts with bash)`	`bash`
Login shell reads	`~/.bash_profile`	`~/.profile`	`first of ~/.bash_profile, ~/.bash_login, ~/.profile`
Full-sudo group	`wheel`	`sudo`	`configured in /etc/sudoers (varies)`
Mandatory access control	`SELinux: getenforce | ausearch -m avc -ts recent`	`AppArmor: sudo aa-status`	`AppArmor: sudo aa-status`